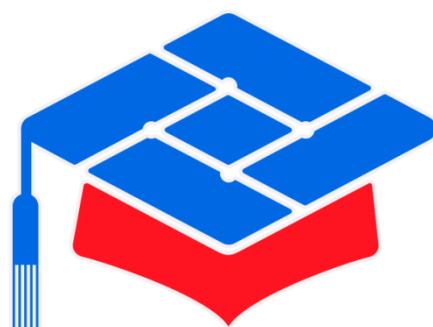




第十届

全国大学生集成电路创新创业大赛

报告类型： 作品报告

参赛杯赛： “飞腾”企业命题

作品名称： AquaGarden——基于飞腾派与多模态 AI 机械臂的居家实时调度水陆生态圈

队伍编号： CICC1003424

团队名称： 你们说的都队

目录

一、技术文档内容预览	4 -
1.1 作品定位	4 -
1.2 系统核心功能	4 -
1.3 基于飞腾派设计亮点	5 -
二、需求分析	7 -
2.1 背景与问题分析	7 -
2.2 总体需求目标	7 -
2.3 用户痛点分析	7 -
2.4 功能需求列表	8 -
2.5 性能需求列表	9 -
2.6 竞品分析	10 -
三、系统架构设计	13 -
3.1 架构设计原则	13 -
3.2 系统总体架构	13 -
3.3 主要模块定义	14 -
3.4 模块接口关系	15 -
3.5 系统数据流	16 -
四、系统详细设计	16 -
4.1 飞腾派主控子系统详细设计	16 -
4.2 OpenAMP/RPMsg 通信设计	17 -
4.3 SPI 通信协议详细设计	18 -
4.3.1 命令帧格式	18 -
4.3.2 响应帧格式	19 -
4.4 命令接口定义	20 -
4.5 物理信号连线设计	20 -
4.6 传感器与执行器子模块设计	21 -
4.7 自动灌溉与水循环流程设计	21 -
4.8 鱼类行为监测	22 -
4.9 植物状态分析设计	23 -
4.10 机械臂任务调度设计与算法	23 -
4.11 多模态生态状态分析	23 -
4.12 Web 平台与大模型交互设计	23 -
4.12.1 整体架构	24 -
4.12.2 后端与通信	24 -
4.12.3 前端控制台	24 -
五、系统设计方案实现	24 -
5.1 实现概述	24 -
5.2 硬件实现	25 -
5.2.1 飞腾派主控层实现	25 -
5.2.2 飞腾派与外设节点的 SPI 通信实现	26 -
5.2.3 SPI 通信参数与帧传输设计	26 -
5.2.4 OpenAMP 裸机从核通信实现	27 -

5.3 算法实现	- 28 -
5.3.1 视觉模型部署	- 28 -
5.3.2 三项机械臂任务实现	- 28 -
5.3.3 大模型交互实现	- 29 -
5.4 软件实现	- 29 -
5.4.1 后端服务实现	- 29 -
5.4.2 前端控制台实现	- 30 -
5.4.3 机械臂 Agent 实现	- 30 -
六、风险分析与管控	- 30 -
6.1 风险分析思路	- 30 -
6.2 风险项与管控措施	- 30 -
6.3 重点风险管控说明	- 31 -
七、系统测试与分析	- 32 -
7.3 功能测试表	- 33 -
7.4 通信链路测试分析	- 34 -
7.5 传感器与执行器测试分析	- 35 -
7.5.1 土壤湿度传感器模块测试	- 35 -
7.5.2 环境温湿度传感器模块测试	- 36 -
7.5.3 水下温度传感器模块测试	- 38 -
7.5.4 水泵驱动模块测试	- 41 -
7.5.5 FreeRTOS 联调测试结果	- 42 -
7.6 算法测试	- 43 -
7.6.1 视觉检测测试	- 43 -
7.6.2 机械臂任务成功率测试	- 44 -
7.7 软件测试	- 44 -
7.7.1 接口测试	- 44 -
7.7.2 前端交互测试	- 44 -
7.8 测试结论	- 45 -
参考文献	- 45 -

一、技术文档内容预览

1.1 作品定位

作品名称：AquaGarden——基于飞腾派与多模态 AI 机械臂的居家实时调度水陆生态圈

AquaGarden 是一套面向居家场景的智能水陆生态系统。Aqua 取自 Aquarium（水族馆）的首部，Garden 对应盆栽花园，系统将鱼缸、盆栽、机械臂、传感器网络、视觉和识别、边缘计算 Web 控制平台集成在同一装置中，通过鱼缸废水循环灌溉盆栽，实现“水体养殖—植物吸收—智能监测—自动执行”的闭环调度。

本作品核心技术路线以飞腾派 PhytiumPi 为系统主控平台。飞腾派承担边缘计算、通信调度、数据融合、任务管理、Web 服务和 AI 推理等核心职责；OpenAMP 裸机从核负责实时 SPI 通信；底层传感器与水泵、灯光、雾化等执行器由外设节点完成采集和驱动。该作品突出了飞腾派在多核异构调度、边缘智能、核间通信、物联网网关与实时控制链路中的核心作用。

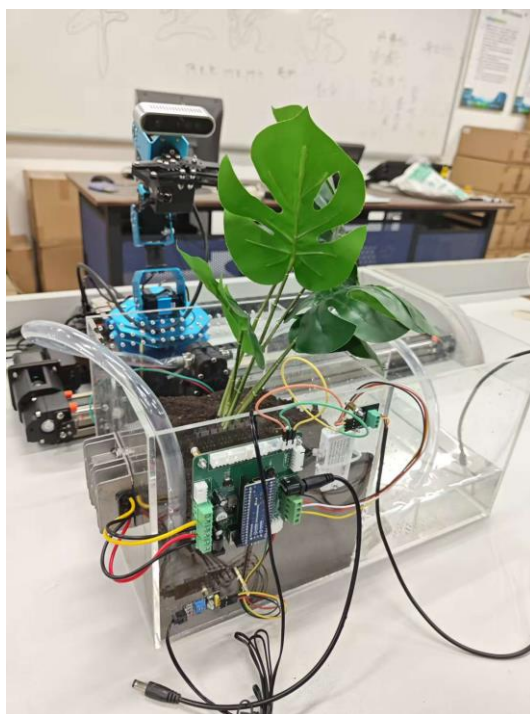


图 1-1 AquaGarden 系统整体实物图

1.2 系统核心功能

系统围绕“智能感知、自动调度、机械执行、远程交互”四个层次设计，主要功能包括：

第一，**水质与环境监测**。系统采集鱼缸水温、pH、浑浊度、溶解氧等指标，同时采集盆栽端土壤湿度、环境温湿度、光照等数据。飞腾派周期性获取底层传感数据，并在 Web 端形成实时数据卡片和历史曲线，为水体健康和植物状态判断提供依据。

第二，**鱼菜共生水循环**。鱼缸中的富营养废水不直接排放，而是通过水泵输送至盆栽端灌溉，植物吸收其中的氮、磷等营养物质。系统根据水质阈值、土壤湿度和运行模式决定是否触发换水、灌溉或循环，提高资源利用率并降低人工维护成本。

第三，**机械臂辅助养护**。系统配置机械臂，可执行投食辅助、松土、叶片修剪等任务。飞腾派负责接收任务请求、分析传感器与视觉信息、生成任务调度指令，再将具体控制命令下发至执行层。

第四，**深度学习鱼类监测**基于 YOLOv8[7] 实现。固定在鱼缸上方或侧面的摄像头实时采集画面，模型对缸内鱼类进行品种识别和运动轨迹跟踪。系统会持续分析鱼类的行为模式，比如是否长时间静止、是否出现翻肚、是否异常聚集在某个角落，结合水质数据综合判断鱼类的健康状态。一旦检测到异常，立即触发告警并记录当时的传感器数据和视频片段，方便用户事后复查[6]。

第五，**Web 端远程监控**。用户可通过浏览器查看传感器数据、视频画面、机械臂任务状态、告警信息和历史记录，也可远程触发水泵、补光、雾化、投食、松土等操作。系统通过 WebSocket 或轮询机制保证数据刷新及时性。



图 1-2 系统核心功能总览图

1.3 基于飞腾派设计亮点

本作品的飞腾派版本重点体现以下设计亮点：

一是采用**飞腾派作为系统主控**。飞腾派不只是普通上位机，而是系统的数据中心、任务调度中心和边缘智能中心。其 Linux 主核运行业务后端、Web 服务、AI 推理与任务管理程序；裸机从核运行 OpenAMP 远程核固件，专职承担 SPI 物理通信与时序控制。

二是**构建 OpenAMP + RPMsg + SPI 的实时通信链路**。Linux 业务进程不直接操作 SPI，而是通过 RPMsg 将 64 字节命令帧交给裸机从核，由裸机从核驱动 FSPIO 与底层外设节点通信。该设计符合飞腾杯对飞腾派多核异构能力和 OpenAMP 使用场景的要求。飞腾派参考方案中明确采用“Linux 主核 + OpenAMP remote core + RPMsg + SPI Master”的架构，SPI 物理层使用 4 线 SPI、64 字节定长帧，并以 v2 路径作为任务对齐版本。

三是保留双路径容错机制。系统默认采用 v2 OpenAMP/RPMsg 路径；当远程核或 RPMsg 链路不可用时，可切换到 v1 spidev fallback 路径，保证比赛演示和工程调试的稳定性。飞腾派使用说明中也将 v2 作为默认路径，v1 作为 fallback，并说明两者协议字节流一致、业务客户端无需感知差异。



图 1-3 飞腾派在系统中的核心作用图

四是实现“感知—判断—执行—反馈”闭环。系统不是单纯采集传感器数据，也不是单纯远程控制设备，而是通过飞腾派将传感器、视觉、机械臂、Web 平台和大模型交互统一调度，实现自动换水、自动灌溉、异常告警、机械臂任务执行和用户反馈闭环。

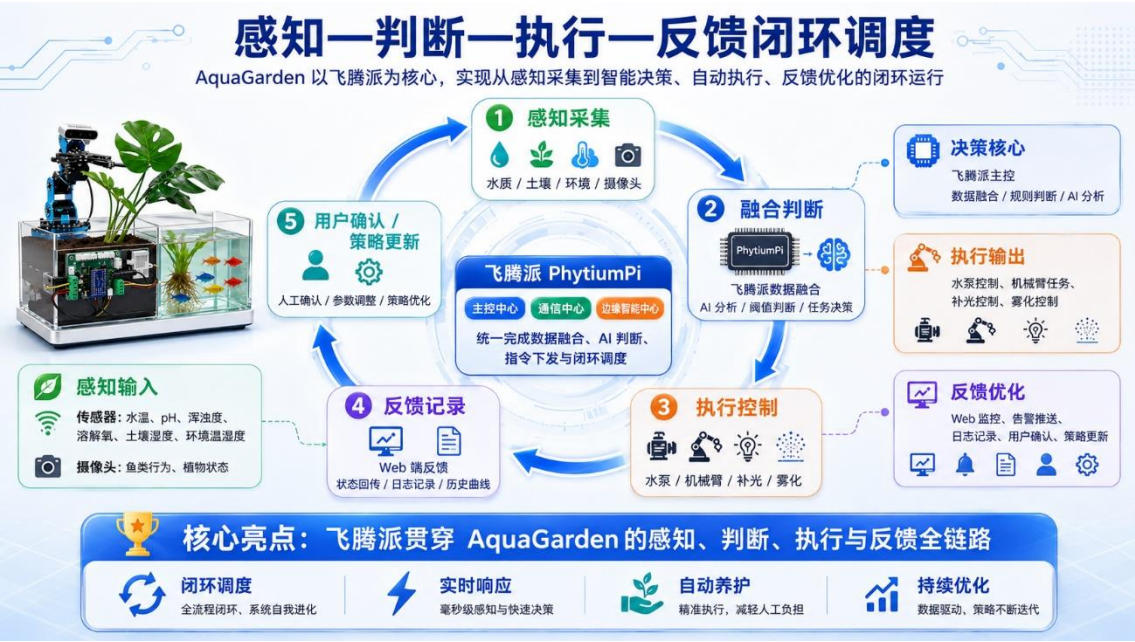


图 1-4 感知—判断—执行—反馈闭环图

二、需求分析

2.1 背景与问题分析

做这个项目，最初的触发点其实很日常。团队里有人在宿舍养了一缸鱼，断断续续养了将近一年，最大的感受是：养鱼这件事对时间规律的要求很高。水要定期换，pH 要隔几天测一次，喂食不能太随意，出去玩几天回来水可能已经发白发臭。和鱼一起死掉的，还有一盆没人浇水的绿萝。

把这两件事放在一起想了一下，发现问题的结构其实很像：两者都属于“需要规律性人工干预、但现有自动化工具很不完善”的类别。更有意思的是，鱼缸换出来的废水其实是很好的植物肥料——鱼类排泄物中的氨氮、磷等成分，正是植物生长所需的营养元素。大多数人换水都是直接倒掉，这是一个很典型的资源浪费。

这两个观察合在一起，就有了 AquaGarden 的基本想法：做一套能自动调度的水陆一体系统，把鱼缸的废水利用起来给植物浇水，同时用机械臂处理两侧都需要的体力活（喂食、修剪、清洁），让用户真正做到“买来放在那里，系统自己跑”。

现有智能鱼缸多集中在水温监测、定时喂食和 App 控制，缺少植物端集成与资源循环利用；传统鱼菜共生套件虽然具备水循环结构，但自动化和智能化程度较低；科研级鱼菜共生系统通常传感器配置较完善，但缺少面向家庭用户的交互设计和执行机构。AquaGarden 的目标是把鱼缸与盆栽真正打通，并通过飞腾派实现边缘智能调度，使系统具备可监测、可判断、可执行、可远程管理的完整能力。

2.2 总体需求目标

系统需要满足以下总体目标：

第一，**实时感知**。系统应连续采集水质、土壤、空气环境和设备状态数据，保证数据刷新满足用户观察和自动控制需要。

第二，**自动决策**。系统应根据阈值规则、历史数据和 AI 分析结果判断是否需要换水、灌溉、补光、雾化、投食或触发告警。

第三，**可靠通信**。飞腾派与底层外设节点之间应采用稳定的 SPI 通信协议，支持命令下发、状态返回、传感器数据读取、错误校验和超时处理。

第四，**机械执行**。系统应支持机械臂完成至少一种典型自动化养护任务，并具备扩展到投食、松土、清理和修剪任务的能力。

第五，**友好交互**。用户应能通过 Web 端查看数据、控制设备、查看告警和触发自动任务，系统应提供清晰的状态反馈。

第六，**工程可扩展**。系统应具备后续增加新传感器、新执行器、新命令和新 AI 模型的能力，避免比赛作品停留在一次性 Demo。

2.3 用户痛点分析

通过对身边养鱼、养植物用户的观察和访谈，我们总结出以下几类核心痛点，以及 AquaGarden

对应的解决方式（见表 2-1）：

表 2-1 用户痛点与解决方案对应表

用户痛点	具体表现	AquaGarden 解决方式
维护耗时，容错率低	需定期换水、测 pH、喂食，出门几天鱼就可能死亡	四路传感器实时监测+水泵自动换水+机械臂定时投食
异常发现滞后	水质恶化或鱼生病，肉眼看到时已错过处理时机	传感器超阈值+YOLOv8 行为分析双重预警，3 秒内推送
废水浪费	换出的鱼缸水直接倒掉，浪费氮磷钾等营养成分	废水自动输送至盆栽端灌溉，变废为肥
植物无人照料	出门植物干死，定时浇水设备不判断土壤实际状态	土壤湿度传感器驱动按需灌溉，不依赖定时
操作门槛高	鱼菜共生套件需专业知识，水质失衡容易全盘崩溃	全自动调度+Web 端可视化+语音自然语言控制

2.4 功能需求列表

表 2-2 功能需求列表

需求编号	功能模块	功能需求	详细定义	关键性能指标	优先级
FR-01	水质监测	采集鱼缸水质数据	周期性采集水温、pH、浑浊度、溶解氧等数据，并上传至飞腾派主控	数据刷新周期 $\leq 5\text{ s}$ ；异常识别延迟 $\leq 3\text{ s}$	高
FR-02	土壤监测	采集盆栽土壤状态	获取土壤湿度、土壤状态或养分相关数据，用于灌溉判断	土壤湿度采集周期 $\leq 5\text{ s}$	高
FR-03	环境监测	采集空气温湿度与光照	获取盆栽周围环境数据，用于补光、雾化和生长状态分析	环境数据刷新周期 $\leq 5\text{ s}$	中
FR-04	水泵控制	控制水循环和灌溉	支持水泵开关、PWM 强度和自动循环策略	控制响应时间 $\leq 10\text{ s}$ ；PWM 可配置	高
FR-05	自动灌溉	按土壤状态触发灌溉	当土壤湿度低于阈值时，系统自动将鱼缸水输送至盆栽端	误触发率低；灌溉触发延迟 $\leq 10\text{ s}$	高
FR-06	自动换水	实现鱼缸水循环维护	根据水质状态触发换水或循环，提高水体稳定性	换水流程可远程启动/停止	高
FR-07	设备告警	监测异常并推送告警	水温、浑浊度、溶解氧、土壤湿度等超阈值时生成告警	告警响应时间 $\leq 3\text{ s}$	高
FR-08	鱼类行为分析	识别鱼类异常状态	通过视觉模型分析鱼类活动轨迹、静止、聚集等行为	常见鱼类识别准确率 $\geq 90\%$	中
FR-09	植物状态	识别黄叶或异常	通过视觉模型检测植物叶片	黄叶识别率 $\geq 85\%$	中

需求编号	功能模块	功能需求	详细定义	关键性能指标	优先级
	分析	叶片	颜色异常,用于修剪提示或机械臂任务		
FR-10	机械臂投食	辅助完成定时投食	系统按时间或用户指令触发机械臂投食动作	单次任务成功率 $\geq 90\%$	中
FR-11	机械臂松土	执行盆栽松土动作	机械臂按预设轨迹在盆栽土面执行松土	路径执行误差 $\leq 5\text{ mm}$	中
FR-12	机械臂清理	清理水面漂浮物	识别漂浮物后调度机械臂进行清理	识别后可生成任务指令	中
FR-13	飞腾派主控调度	统一管理采集、控制和 AI 任务	飞腾派运行后端、AI、通信守护进程和 Web 服务	系统连续运行 $\geq 72\text{ h}$	高
FR-14	OpenAMP 通信	实现 Linux 裸机从核通信	Linux 业务层通过 RPMsg 将命令发送至裸机从核	RPMsg 通道可稳定收发 64B 帧	高
FR-15	SPI 通信协议	飞腾派与外设节点通信	裸机从核作为 SPI Master, 底层节点作为 SPI Slave, 完成命令与响应	SPI Mode 0; 1 MHz; 64B 定长帧; CRC 校验	高
FR-16	Web 可视化	展示实时数据与历史曲线	展示传感器数据卡片、趋势图、视频和设备状态	前端刷新间隔 $\leq 2\text{ s}$	高
FR-17	远程控制	支持用户手动控制设备	Web 端可控制水泵、灯光、雾化、机械臂任务	指令响应时间 $\leq 3\text{ s}$	高
FR-18	大模型交互	自然语言查询生态状态	用户可询问“鱼缸状态如何”等问题,系统生成自然语言分析	支持状态摘要生成	中
FR-19	数据记录	保存历史运行数据	保存传感器数据、告警事件、机械臂任务日志	支持按时间查询	中
FR-20	故障自恢复	通信异常后自动恢复	对 CRC 错误、超时、SPI 失败进行统计和重试	连续错误可触发链路重置	高

2.5 性能需求列表

表 2-3 性能需求列表

指标类别	指标项	设计目标	说明
传感采集	水质数据刷新周期	$\leq 5\text{ s}$	满足实时监测与告警判断
通信链路	SPI 单帧长度	64 Byte	定长帧便于抗错位和 DMA/缓存对齐

指标类别	指标项	设计目标	说明
通信链路	SPI 模式	Mode 0	CPOL=0, CPHA=0
通信链路	SPI 速率	首发 1 MHz, 可扩展至 4 MHz	兼顾稳定性与实时性
通信链路	通信校验	CRC16	防止帧错误、粘包和错位
主控调度	传感器轮询周期	250 ms 级缓存更新	飞腾派守护进程周期性拉取状态
控制响应	水泵/执行器响应时间	≤ 10 s	从阈值触发到动作执行
Web 显示	数据可视化刷新闻隔	≤ 2 s	WebSocket 或轮询刷新
AI 识别	鱼类识别准确率	$\geq 90\%$	正常光照、常见观赏鱼条件下
AI 识别	黄叶识别率	$\geq 85\%$	正常室内光照条件下
机械臂	重复定位误差	$\leq 2 - 5$ mm	与机械结构和标定精度相关
稳定性	无人值守运行时长	≥ 72 h	面向居家场景连续运行
可靠性	通信异常处理	支持重试、统计与 fallback	保证演示与长期运行稳定

2.6 竞品分析

目前市场上与本作品方向相关的产品主要分为四类：成熟的商业智能鱼缸、基础款智能鱼缸、传统鱼菜共生套件，以及学术/科研级系统。以下从多个维度进行对比分析。

当贝智能鱼缸 2S Ultra（2026 年发布）：目前商业化程度最高的智能鱼缸，具备 AI 识鱼、水质和水温检测、智能喂食、App 远程控制等功能，外观设计和用户体验较为成熟，参考价约 2000 元。但该产品没有机械臂，没有植物端集成，无法实现鱼菜共生，也没有大模型自然语言交互能力，本质上是一个“带 AI 的传统鱼缸”。

米家智能桌面鱼缸（2025 年众筹上市，众筹价 279 元）：主打轻量化和低价，支持定时喂食、水温监测和米家 App 控制，适合小空间使用。功能比较基础，更接近“带 App 的普通鱼缸”，与 AquaGarden 的定位差异较大。

Potti 鱼菜共生系统（轻巧版）：来自香港的商业产品，体积小（27×27×25 cm），5 升鱼缸配 0.5 升种植盆，内置虹吸循环和 3W 静音水泵，卖点是“五分钟安装”。但该产品完全没有传感器和自动化功能，水质、湿度全靠人工判断，没有 App 或远程监控，本质上还是一个手动鱼菜共生套件，只是体积更小更好看。同类产品还有台湾的 AquaSmart 桌上型鱼菜共生系统和 Lagoon Vish 2.0，功能层面大同小异，均停留在“物理结构+水循环”阶段，自动化程度极低。

CyPhA (Cyber Physical Aquaponics) 系统[3]：2023 年发表于学术期刊的研究级鱼菜共生系统，采用信息物理融合架构，集成了 pH、溶解氧、TDS、水温、气温、湿度六路传感器，通过 MQTT 协议通信，并构建了包含生化过滤、垂直种植的完整五级水处理结构。传感器配置完善，逻辑严谨，但该系统没有机械臂，没有视觉识别，没有面向家庭用户的交互设计，

属于实验室验证性系统，不对外销售。同期发表在《Harbin Engineering University Journal》上的“Integrated Aquaponics Through Smart Sensors”[4] 也是类似的研究方向，该系统通过 IoT 传感器网络实现了鱼类健康评分提升 50~60%，但同样停留在传感器监测层面，没有执行机构和视觉智能。

表 2-4 竞品分析

产品	参考图片
当贝 2S Ultra	
米家智能鱼缸	
Potti	

表 2-5 竞品对比分析表

对比维度	AquaGarden (本作品)	当贝智能鱼缸	米家智能桌面鱼缸	Potti/AquaSmart 鱼菜共生套件	CyPhA 等学术系统
水质检测路数	4 路 (pH/温度/浑浊度/溶解氧)	2 路 (水质/水温)	1 路 (水温)	无传感器, 手动检测	5~6 路
自动喂食	✓ 定时+机械臂辅助	✓ 定时	✓ 定时	X	X
多功能机械臂	✓ 6 自由度 (剪叶/松土/清洁/投食)	X	X	X	X
植物端集成	✓ 鱼菜共生全自动	X	X	✓ 手动水循环	✓ 实验级
废水循环利用	✓ 传感器驱动自动灌溉	X	X	✓ 虹吸被动循环	✓ 实验级
鱼类 AI 识别与行为分析	✓ 品种识别+轨迹跟踪+异常报警	✓ AI 识鱼 (品种识别)	X	X	X
大模型自然语言交互	✓	X	X	X	X
叶片自动修剪	✓ YOLO+深度相机全自动	X	X	手动	X
延时摄影	✓	X	X	X	X
Web 端远程监控	✓ 自研 Web 端	✓ App	✓ 米家 App	X	X
家庭用户友好度	高	高	高	中 (需一定经验)	极低 (实验室用)
价格参考	竞赛作品	约 2000 元	约 279 元	300~1500 元	不对外销售

从对比来看,当贝、米家等商业产品在用户体验上已经比较成熟,但本质上都是“传统鱼缸+App 控制”的升级,没有植物端、没有机械臂;Potti、AquaSmart 这类鱼菜共生套件有植物端,但停留在纯物理结构层面,连最基本的传感器都没有,自动化几乎为零;CyPhA 等学术系统传感器配置扎实,但没有执行机构,只能“看”不能“做”,更没有面向家庭的产品化设计。AquaGarden 是目前唯一同时具备多功能机械臂、鱼菜共生自动调度、多路传感监测、视觉 AI 和大模型交互的系统,在功能整合度上与现有产品拉开了明显差距

三、系统架构设计

3.1 架构设计原则

系统架构遵循“**飞腾派主控、底层外设协同、通信协议统一、业务与实时解耦**”的设计原则。

第一，**主控集中化**。飞腾派作为系统唯一的主控中心，统一负责传感器数据汇聚、设备控制调度、视觉与 AI 分析、Web 服务和运行日志管理。这样可以避免多个主控之间职责不清，降低系统联调复杂度。

第二，**实时链路独立化**。飞腾派 Linux 主核运行业务程序，适合处理 Web、数据库、AI 推理等复杂任务；裸机从核通过 OpenAMP 独立运行 SPI 通信固件，避免 Linux 调度抖动直接影响底层通信时序。

第三，**底层执行节点轻量化**。RA6E2 或其他外设节点不承担全局调度，只负责传感器读取、水泵/灯光/雾化等执行器驱动和状态反馈。系统逻辑尽量上收至飞腾派，便于后续扩展和比赛展示。

第四，**协议标准化**。飞腾派与底层节点之间采用统一 SPI 帧协议，包含帧头、版本、序列号、设备类型、命令字、数据长度、负载和 CRC 校验。新增设备或新增命令时，只需要扩展协议定义和两端解析逻辑。

3.2 系统总体架构

系统从上到下分为五层：

- (1) **用户交互层**：Web 控制台、移动浏览器访问页面、大模型对话入口；
- (2) **应用服务层**：Spring Boot 后端、Vue 前端、AI 分析服务、机械臂任务调度服务；
- (3) **飞腾派主控层**：Linux 主核、aqua_rpmsgd 守护进程、OpenAMP/RPMsg 通道、裸机从核固件；
- (4) **外设执行层**：RA6E2 外设节点、STM32 机械臂执行控制器、传感器和执行器驱动；
- (5) **生态对象层**：鱼缸水体、鱼类、盆栽土壤、植物、机械臂末端工具。

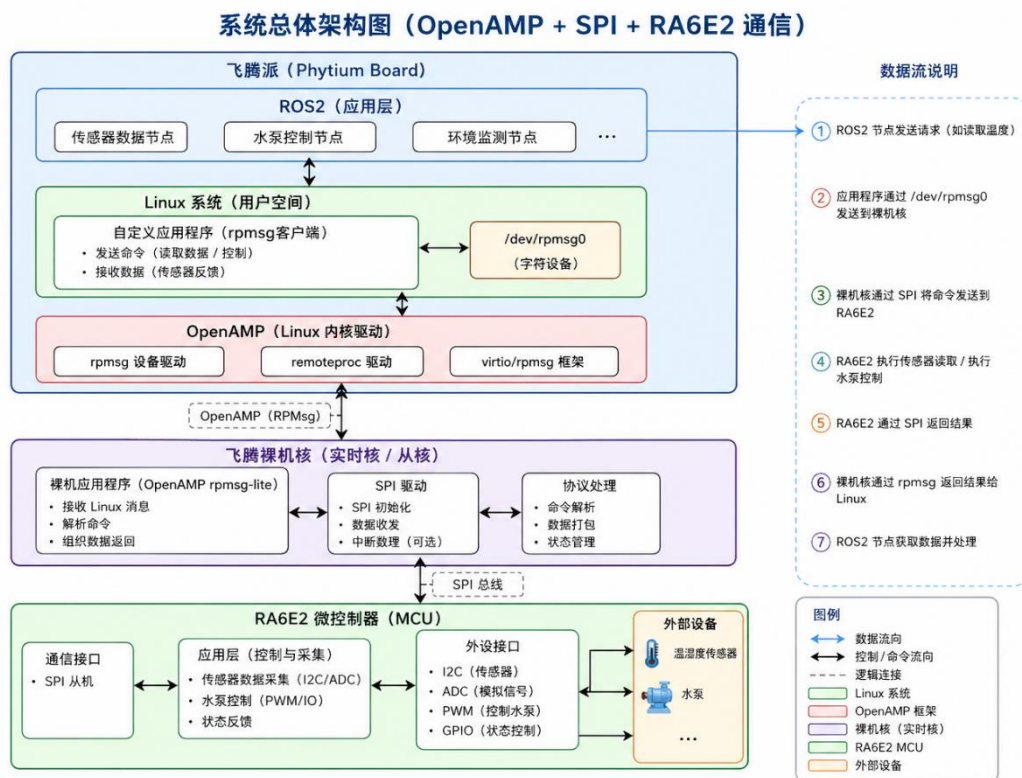


图 3-1 AquaGarden 系统总体架构图

3.3 主要模块定义

表 3-1 主要模块定义汇总表

模块名称	所属层级	主要职责	输入	输出
Web 控制台	用户交互层	展示数据、视频、告警和控制按钮	后端 API / WebSocket	用户控制指令
后端服务	应用服务层	数据存储、接口管理、任务管理	传感器数据、用户请求	控制命令、告警信息
AI 分析模块	应用服务层	鱼类行为、植物状态、生态摘要分析	视频帧、历史数据	识别结果、自然语言建议
飞腾派 Linux 主核	主控层	运行后端、AI、通信守护和业务逻辑	用户请求、RPMsg 数据	RPMsg 命令、Web 数据
aqua_rpmsgd	主控层	IPC 与 RPMsg 转发、帧打包/解析、状态缓存	Unix Socket 请求	64B CMD/RSP 数据
OpenAMP 裸机从核	主控层	RPMsg ↔ SPI 字节透传、SPI 时序控制	64B 命令帧	64B 响应帧
RA6E2 外设节点	外设执行层	传感器采集、水泵/灯光/雾化控制	SPI 命令	传感器数据、执行状态

模块名称	所属层级	主要职责	输入	输出
STM32 机械臂控制器	外设执行层	舵机驱动、机械臂动作执行	机械臂任务指令	执行状态
传感器组	生态对象层	水质、土壤、环境数据采集	生态状态	原始测量数据
执行器组	生态对象层	水泵、灯光、雾化、机械臂动作执行	控制信号	物理动作

3.4 模块接口关系

系统的接口关系分为三类。

第一类是用户与飞腾派之间的应用接口。用户通过 Web 页面访问飞腾派后端服务，后端提供登录、传感器数据查询、设备控制、历史曲线、机械臂任务和大模型问答等接口。该层接口强调易用性和可视化。

第二类是飞腾派 Linux 主核与裸机从核之间的核间通信接口。Linux 业务程序或 aqua_rpmgsd 守护进程通过 Unix Domain Socket 接收业务命令，再通过 RPMsg 发送至 OpenAMP 裸机从核。飞腾派参考方案中，Linux 业务进程通过 /tmp/aqua_spi.sock 与守护进程通信，守护进程再经 /dev/rpmgsN 与裸机核通信。

第三类是飞腾派与底层外设节点之间的 SPI 物理接口。裸机从核驱动 FSPIO，以 SPI Master 身份向 RA6E2 从机发送命令帧。SPI 采用 4 线连接，包含 SCK、MOSI、MISO、CS 和 GND，飞腾派使用 SPI0 排针，RA6E2 对应 P100-P103 引脚。飞腾派使用说明中明确给出了该物理连线表，并要求短线连接和共地。

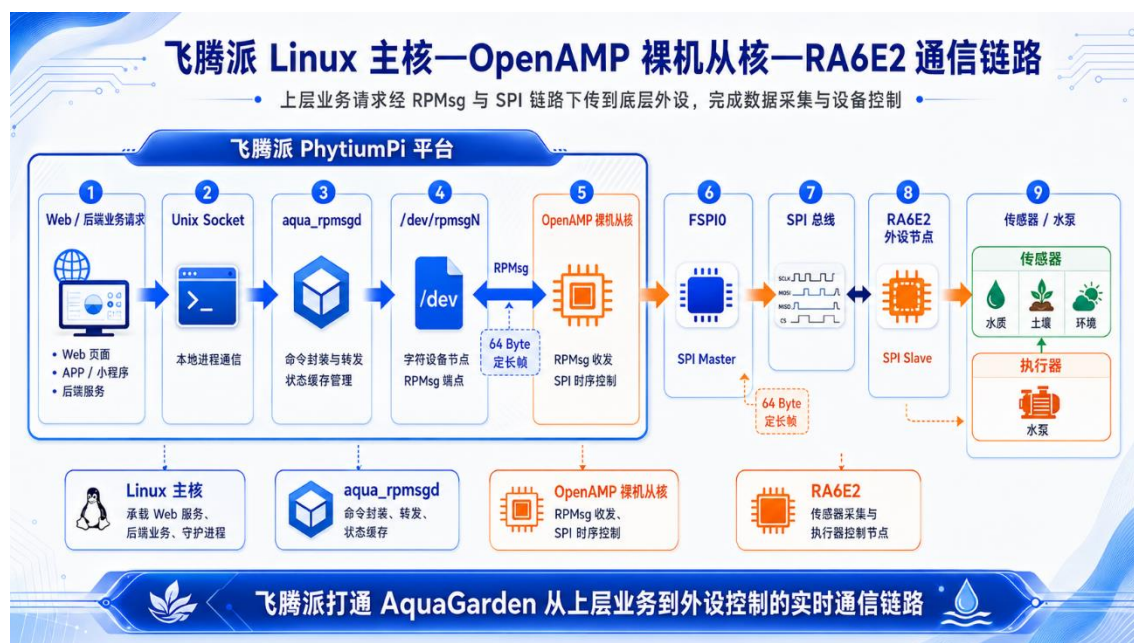


图 3-2 飞腾派 Linux 主核—OpenAMP 裸机从核—RA6E2 SPI 通信链路图

3.5 系统数据流

系统数据流可概括为“采集上行”和“控制下行”两条路径。

采集上行路径：

传感器 → RA6E2 外设节点 → SPI 响应帧 → 飞腾派裸机从核 → RPMsg → aqua_rpmsgd → 后端服务 → Web 前端 / AI 分析模块。

控制下行路径：

用户指令 / AI 决策 → 后端任务管理 → aqua_rpmsgd → RPMsg → OpenAMP 裸机从核 → SPI 命令帧 → RA6E2 / 机械臂控制器 → 执行动作 → 状态反馈。

这种架构的优点是数据流清晰，实时通信链路和上层业务逻辑边界明确。即使 Web 页面、AI 模型或数据库短时异常，底层通信链路仍可通过守护进程和裸机核保持基本控制能力；若 OpenAMP 链路出现问题，也可通过 fallback 路径保障调试和演示。

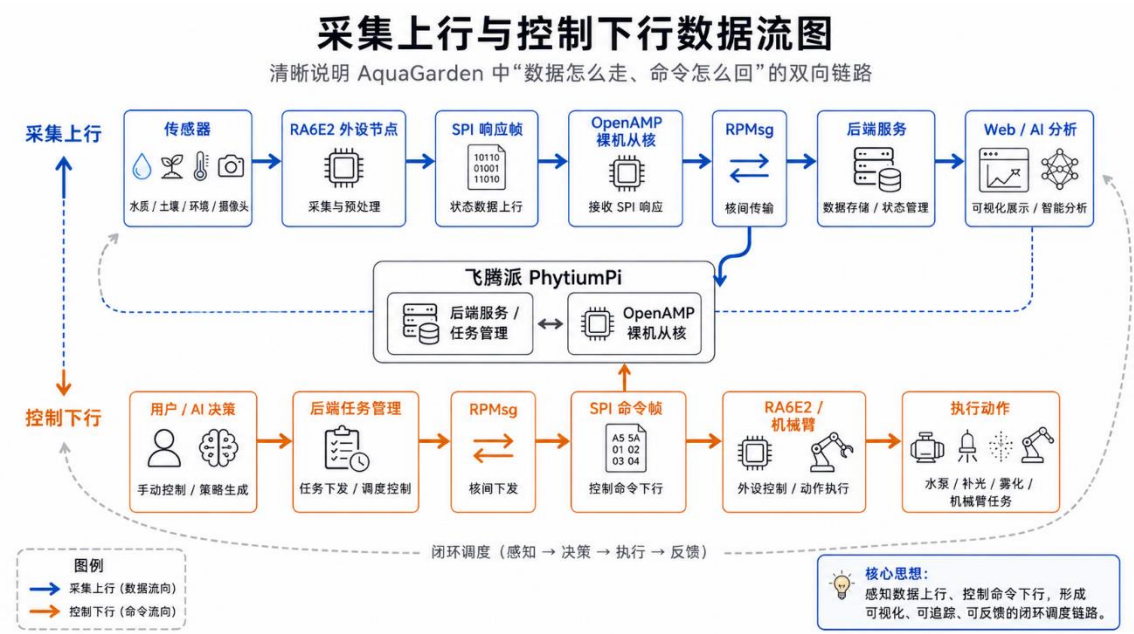


图 3-3 系统数据流图

四、系统详细设计

4.1 飞腾派主控子系统详细设计

飞腾派主控子系统由 Linux 主核和 OpenAMP 裸机从核共同构成。Linux 主核主要运行四类程序：后端服务、Web 前端服务、AI 推理与任务调度服务、aqua_rpmsgd 通信守护进程。裸机从核运行 openamp_spi_core0.elf 固件，负责接收 RPMsg 命令、驱动 SPI0、维持片选时序、执行 64 字节命令帧与响应帧的透传。

该设计的核心是“业务逻辑与实时通信分离”。Linux 主核适合运行复杂业务，但存在进程调度、文件系统、网络服务等不确定因素；裸机从核无 Linux 进程调度干扰，适合承担 SPI

时序控制。飞腾派 v2 方案中明确将 Linux 主核用于业务层，将裸机从核用于 OpenAMP remote core 和 SPI Master 物理驱动，二者通过 RPMsg 连接。



图 4-1 飞腾派主控子系统内部结构图

4.2 OpenAMP/RPMsg 通信设计

Linux 主核与裸机从核之间使用 RPMsg 通道，服务名为 aqua-spi。Linux 端守护进程不固定写死 /dev/rpmstd0，而是通过系统节点扫描方式查找对应的 RPMsg 设备，避免多个 RPMsg 服务同时存在时设备编号漂移造成通信失败。

RPMsg 通信采用“一个请求对应一个响应”的同步模型。Linux 端把 SPI 命令封装为 64 字节命令帧，通过 RPMsg 发送给裸机从核；裸机从核收到命令帧后，先向 SPI 总线发送该命令，再等待约 5 ms，让底层从机准备响应；随后裸机从核发送 NOP_READ 帧读取本次响应，并将 64 字节响应帧通过 RPMsg 返回 Linux 端。该设计使 Linux 业务层无需理解 SPI 全双工延迟响应细节，只看到“发送命令—收到结果”的简洁接口。

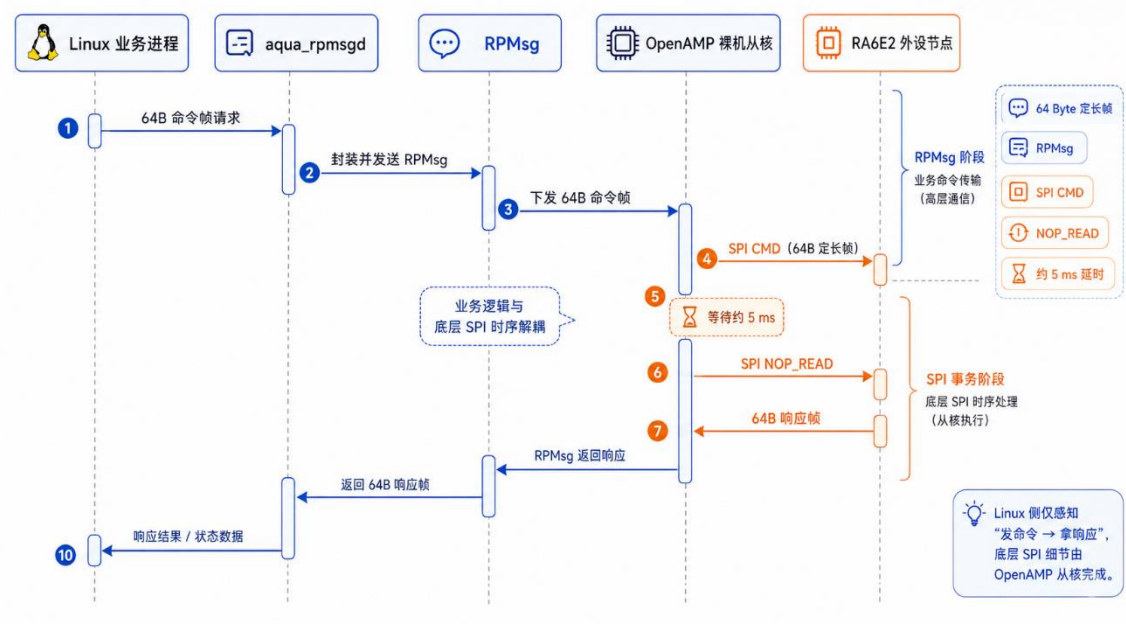


图 4-2 RPMsg 请求—SPI 事务—RPMsg 响应时序图

4.3 SPI 通信协议详细设计

系统采用 4 线 SPI，飞腾派为 Master，RA6E2 外设节点为 Slave。SPI 参数如下：

表 4-1 SPI 参数设计表

参数	设计值	设计理由
SPI 模式	Mode 0, CPOL=0, CPHA=0	与底层从机配置兼容，调试简单
通信速率	首发 1 MHz，可扩展至 4 MHz	兼顾飞腾派排针连接稳定性与实时性
数据位宽	8 bit	按字节流组织协议，避免 16 bit 大小端问题
帧长度	64 Byte 定长帧	抗错位、便于缓存对齐和协议解析
校验方式	CRC16/Modbus	提高通信可靠性
片选管理	每帧一次 CS 边沿	通过 CS 上升沿辅助从机判断帧结束

4.3.1 命令帧格式

表 4-2 命令帧格式汇总表

偏移	长度	字段	说明
0	1	SOF0	帧头 0xA5
1	1	SOF1	帧头 0x5A
2	1	VER	协议版本

偏移	长度	字段	说明
3	1	SEQ	序列号
4	1	DEV	设备类别
5	1	CMD	命令字
6	1	LEN	负载长度
7	1	FLAGS	标志位
8	16	PAYLOAD	命令参数
24	38	RESERVED	保留字段
62	2	CRC16	对前 62 字节校验

4.3.2 响应帧格式

表 4-3 响应帧格式汇总表

偏移	长度	字段	说明
0	1	SOF0	响应帧头 0x5A
1	1	SOF1	响应帧头 0xA5
2	1	VER	协议版本
3	1	ACK_SEQ	对应命令序列号
4	1	STATUS	执行状态
5	1	RSP_TYPE	响应类型
6	1	LEN	响应负载长度
7	1	FLAGS	响应标志
8	48	PAYLOAD	响应数据
56	4	UPTIME_MS	从机运行时间
60	2	RESERVED	保留字段
62	2	CRC16	对前 62 字节校验

该协议通过定长帧、帧头、序列号和 CRC16 同时解决粘包、错位、误码和响应匹配问题。设备类别 DEV 与命令字 CMD 分离，使系统后续可扩展更多传感器、执行器和联动规则，而不需要重构整体协议。飞腾派参考设计中也明确说明 v2 完全复用协议层，并将 64 字节帧作为 RPMsg 和 SPI 的基本传输单元。

4.4 命令接口定义

表 4-4 命令接口定义汇总表

设备类别	命令	参数	功能说明
SYSTEM	SYS_NOP	无	空操作，用于读取上一响应
SYSTEM	SYS_PING	无	测试底层节点是否在线
SYSTEM	GET_VERSION	无	获取协议与固件版本
SENSOR	POLL_ALL	无	读取全部传感器数据
SENSOR	POLL_WATER	无	读取水质相关数据
SENSOR	POLL_SOIL	无	读取土壤湿度数据
SENSOR	POLL_AIR	无	读取空气温湿度数据
PUMP	START	水泵编号	开启水泵
PUMP	STOP	水泵编号	关闭水泵
PUMP	SET_PWM	水泵编号、PWM 占空比	调节水泵强度
LINKAGE	SET_RULE	阈值规则	设置自动联动策略
ROBOT	TASK_FEED	任务参数	触发投食任务
ROBOT	TASK_LOOSEN	路径参数	触发松土任务
ROBOT	TASK_CLEAN	目标参数	触发漂浮物清理任务

4.5 物理信号连线设计

飞腾派与底层外设节点采用 SPI0 物理连接。接线设计如下：

表 4-5 物理信号连线设计汇总表

信号	飞腾派端	外设节点端	说明
SCK	SPI0_SCK	P102 / RSPCK1	SPI 时钟
MOSI	SPI0_MOSI	P101 / MOSI1	主发从收
MISO	SPI0_MISO	P100 / MISO1	从发主收
CS	SPI0_CSN0	P103 / SSL10	片选信号
GND	任一 GND	GND	必须共地

布线时要求杜邦线尽量短，建议小于 15 cm，并保证飞腾派与外设节点共地。比赛展示时，

应将该连线关系绘制为独立图，便于评委快速判断飞腾派确实参与底层物理控制链路。

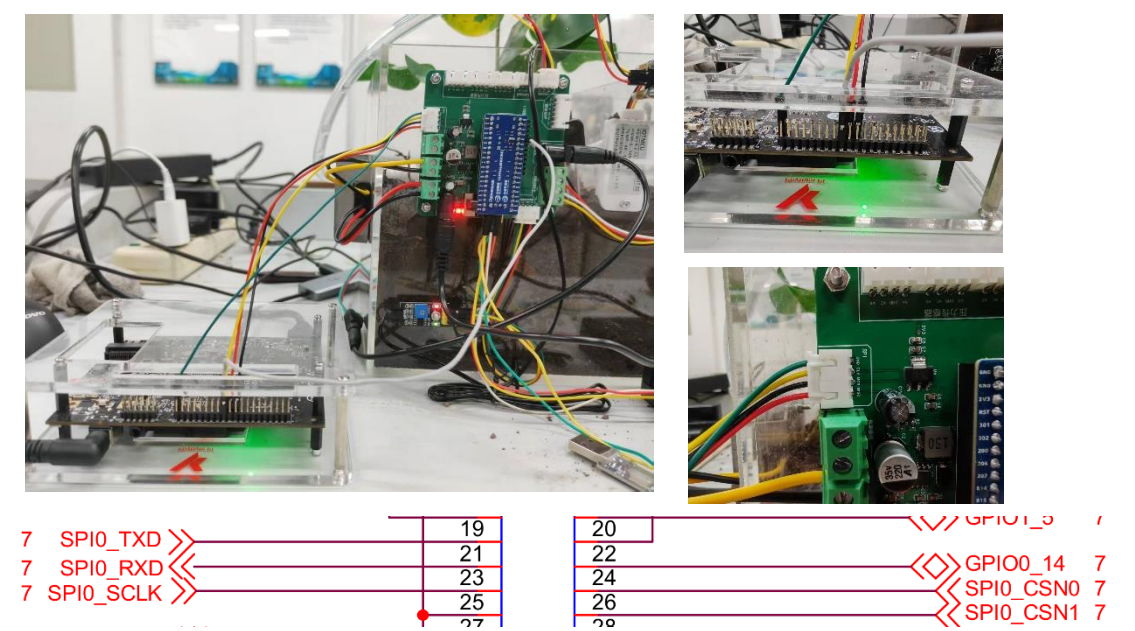


图 4-3 飞腾派 SPI0 与底层外设节点物理信号连线图

4.6 传感器与执行器子模块设计

传感器子模块负责采集生态系统的基础状态。鱼缸端重点监测水温、pH、浑浊度和溶解氧；盆栽端重点监测土壤湿度、环境温湿度和光照。底层节点完成原始采样、基础滤波和越限初判，再通过 SPI 向飞腾派返回结构化数据。

执行器子模块包括水泵、补光灯、雾化模块和机械臂。水泵用于水循环、换水和盆栽灌溉；补光灯用于植物补光；雾化模块用于改善空气湿度；机械臂用于投食、清理、松土和修剪。飞腾派不直接输出 PWM 或舵机波形，而是通过控制命令调度底层执行器，保证高层逻辑与底层电气驱动解耦。

表 4-6 传感器与执行器选型

区域	类型	器件/模块	接口	量程/精度
鱼缸端	水温	DS18B20 防水探头	单总线	-55~125℃，±0.5℃
鱼缸端	pH	pH 电极 + 模拟调理板	ADC	0~14，±0.1
鱼缸端	浑浊度	光电浊度模块	UART	0~1000 NTU
鱼缸端	溶解氧	光学溶解氧模块	UART	0~20 mg/L，±0.2
盆栽端	土壤湿度	电阻式土壤湿度探针	ADC	0~100%
盆栽端	空气温湿度	SHT30	I2C	温度±0.3℃，湿度±2%
两端	执行器	单向水泵、补光 LED、雾化模块	PWM	—

4.7 自动灌溉与水循环流程设计

自动灌溉流程如下：

1. 底层节点采集土壤湿度；
2. 飞腾派周期性读取传感器快照；
3. 后端判断土壤湿度是否低于阈值；
4. 若满足灌溉条件，飞腾派生成水泵控制命令；
5. 裸机从核通过 SPI 将命令发送给外设节点；
6. 外设节点启动水泵，将鱼缸水输送至盆栽端；
7. 系统持续读取土壤湿度和水泵状态；
8. 达到目标湿度或超过安全时间后关闭水泵；
9. 记录灌溉日志并在 Web 端显示结果。

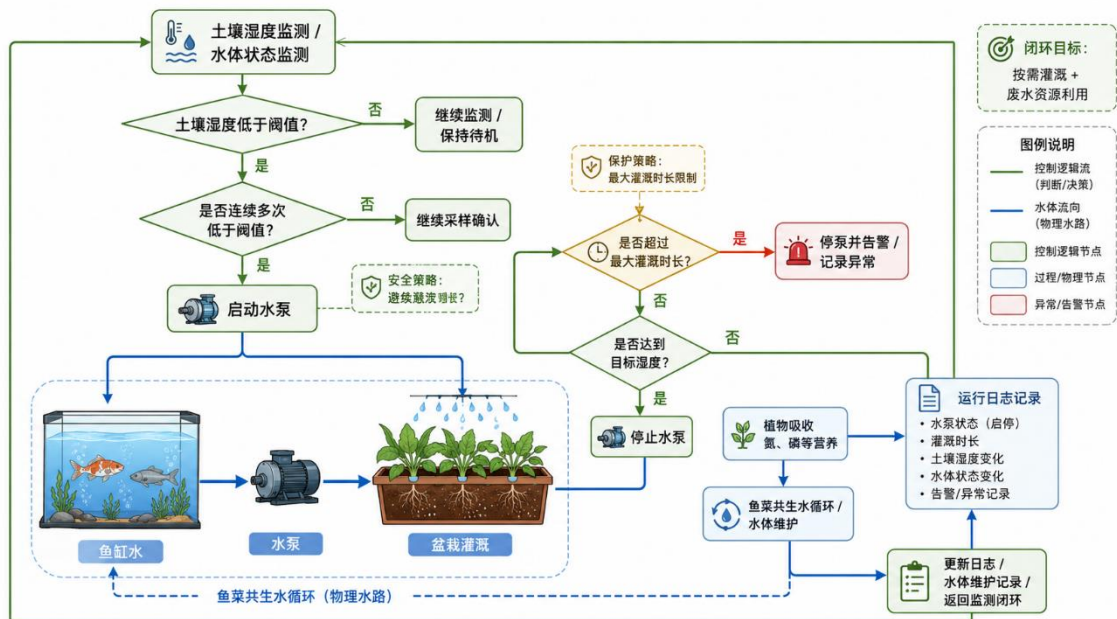


图 4-4 自动灌溉流程图

该流程的关键是安全控制。系统不应只依据单次湿度读数启动水泵，而应采用连续多次低于阈值的判断方式，避免传感器抖动导致误触发。同时需要设置单次最大灌溉时长，防止水泵异常持续运行。

4.8 鱼类行为监测

鱼类行为监测分两个子任务：目标检测和行为分析。

检测部分用 YOLOv8 做实时检测，输入是固定 RGB 摄像头的视频流，输出是每帧图像中鱼的边界框和分类标签。考虑到树莓派 5 的算力，推理时会用 INT8 量化压一下模型体积，目标是在 640×480 分辨率下跑到 15fps 以上。

行为分析部分在检测结果的基础上做：一是轨迹跟踪，用 ByteTrack 给每条鱼分配 ID，追踪帧间位移；二是异常行为判断，主要看两个信号——鱼在某个区域长时间静止不动（可能是患病），以及多条鱼向水面聚集（可能是缺氧）。这两种异常都和水质传感器数据交叉验

证，比如鱼往水面聚集同时溶解氧偏低，可信度就更高，系统会触发告警并自动加快水循环。

4.9 植物状态分析设计

植物状态分析主要面向黄叶识别和生长记录。摄像头周期性采集盆栽图像，系统识别叶片颜色异常区域，并生成修剪建议或机械臂任务。

4.10 机械臂任务调度设计与算法

机械臂任务由飞腾派统一调度，分为自动触发和用户触发两类。自动触发包括定时投食、湿度异常后的松土建议等；用户触发包括 Web 端按钮控制和自然语言指令控制。

（1）自动喂食

喂食用的是一个类似胡椒粉瓶的小型投喂瓶，瓶口有细孔，夹住瓶体倾斜抖动时鱼食会从孔中均匀落下。任务流程如下：

1. 相机先在鱼缸区检测投喂瓶的位置。Aurora 930 输出点云，在点云中提取出瓶子轮廓（形状先验：细长圆柱体），得到瓶体中心在机械臂坐标系下的三维坐标。
2. 规划层计算抓取姿态，夹爪对准瓶体中段，垂直下爪夹住。
3. 机械臂把瓶子提起，滑轨带动整臂横移到鱼缸正上方，末端悬停在水面上约 15cm 处。
4. 执行倒料动作：让末端在 $\pm 30^\circ$ 范围内反复旋转，模拟人工抖动撒料的动作，持续 3-5 秒。具体角度和持续时间根据鱼的数量配置。
5. 完成后机械臂把瓶子放回原位。

（2）松土

松土任务的视觉依赖比较低，主要靠预设轨迹。盆栽区的土壤位置相对固定，在系统初始化时做一次手眼标定，记录土面的位置范围就够了。

1. 触发条件：土壤湿度传感器检测到土层板结特征（湿度长时间低于阈值但近期没有降雨/灌溉触发），或者用户手动触发。
2. 末端换上细杆形松土工具，在土面按 Z 形（“弓字形”）轨迹推进，每隔约 2cm 执行一次插入-提起的动作，深度约 1~2cm。
3. 轨迹由规划层在任务开始前生成一次，执行时按路径点顺序下发，不依赖实时视觉反馈，所以对算力需求很低。

4.11 多模态生态状态分析

单靠传感器数值不太能说清楚水族生态的整体状态，需要结合视觉行为特征一起判断。系统在规划层会定期（比如每 10 分钟）做一次综合评估：把当前的水质传感器读数、鱼类行为分析结果（活跃度评分、异常标志）、盆栽端的温湿度和土壤数据，组装成一段结构化的 prompt，发给 DashScope 的大模型接口，让大模型给出自然语言的健康状态描述和建议操作。

这样用户在前端看到的不是一堆数字，而是“今天鱼的状态比较活跃，水质基本正常，建议明天上午松一下土”这类可读的摘要。语音交互也走这条通路，用户说“今天鱼怎么样了”，后端把最新一次评估结果拼进回答里，TTS 播出来。

4.12 Web 平台与大模型交互设计

4.12.1 整体架构

软件侧采用 B/S 架构，前后端分离。后端是 Spring Boot 3.3（Java 17），提供 REST API、WebSocket 实时推送和 MJPEG 视频流接口；前端是 Vue3 + Vite，单页应用，覆盖登录、控制面板、机械臂控制、历史数据、系统设置等页面。

机械臂 Agent 单独用 FastAPI（Python）部署，负责接收后端的任务指令，调用视觉推理模型，并通过串口或 ROS2 节点把动作下发给树莓派。这样 Spring Boot 不直接跑 Python 机器学习代码，保持语言隔离。

数据存储用 SQLite，适合嵌入式/轻量场景，不需要单独维护数据库服务。

4.12.2 后端与通信

后端核心接口如表 3-4 所示。JWT 做无状态认证，传感器数据接口和机械臂控制接口都需要携带令牌。WebSocket /ws/logs 用于实时推送操作日志，方便用户在前端看到机械臂执行反馈。

表 4-7 后端核心接口

接口	方法	功能描述	是否鉴权
/api/login	POST	用户登录，返回 JWT	否
/api/sensors	GET	获取当前传感器读数（5 类）	是
/api/robot/control	POST	机械臂方向控制指令	是
/api/robot/task	POST	触发喂食/捞叶/松土等自动任务	是
/api/mode	GET/POST	查询或切换运行模式（demo/service）	是
/api/video/tank	GET	鱼缸 MJPEG 视频流	否
/ws/logs	WebSocket	实时日志推送	否

嵌入式侧（RA6E2）通过 UART 把传感器数据上报给树莓派，树莓派侧有一个数据桥接进程把这些数据推送到 Spring Boot 后端的内部接口，再由后端统一对前端提供。

4.12.3 前端控制台

前端按功能划分了几个主要页面：

- （1）**控制面板**：实时展示 5 类传感器数据卡片和双路视频，传感器数据每 5 秒轮询刷新，同时维护历史趋势图。
- （2）**机械臂控制**：支持方向按钮、键盘快捷键、虚拟摇杆三种控制方式；提供喂食、捞叶、松土等预设任务的一键触发按钮；实时展示 WebSocket 日志面板。
- （3）**LLM 交互**：输入框支持文字问询，同时支持语音输入（浏览器 Web Speech API），把生态状态查询结果以对话形式展示。
- （4）**历史数据**：展示传感器历史曲线，支持时间范围筛选。

五、系统设计方案实现

5.1 实现概述

当前系统处于软硬件并行推进阶段：软件平台原型已经完成并可运行，覆盖登录认证、传感

器展示、机械臂控制、实时日志、LLM 交互等核心流程；硬件侧正在组装和联调。本章说明各模块的具体实现方式，软件部分基于已有代码描述，硬件和算法部分结合当前进度说明思路。

5.2 硬件实现

本系统硬件部分采用“飞腾派主控 + 外设采集执行节点 + 机械臂执行节点 + 多模态感知模块”的分层设计。飞腾派 PhytiumPi 作为整机主控平台，承担系统业务调度、边缘计算、Web 服务、AI 推理、核间通信和底层设备命令管理；RA6E2 作为外设采集与执行控制节点，负责传感器数据采集以及水泵、补光灯、雾化模块等低层执行器驱动；STM32 作为机械臂执行控制节点，负责舵机、夹爪和滑轨机构的实时动作控制。

该硬件架构将复杂计算任务、实时通信任务和低层执行任务进行明确分工。飞腾派侧集中处理数据融合、视觉识别、任务决策和用户交互；OpenAMP 裸机从核负责底层 SPI 通信时序；外设节点负责稳定采集与执行；机械臂节点负责动作闭环控制。通过这种分层方式，系统既具备较强的边缘智能能力，又能保证底层控制链路的实时性和可靠性。

5.2.1 飞腾派主控层实现

飞腾派是本系统的核心控制与计算平台。系统利用飞腾派 PE2204 多核处理能力，将其划分为 Linux 主核和 OpenAMP 裸机从核两个运行域。其中，Linux 主核运行业务后端、Web 控制台、AI 推理程序、机械臂任务调度服务和通信守护进程；OpenAMP 裸机从核运行 openamp_spi_core0.elf 固件，专门负责 RPMsg 与 SPI 总线之间的实时数据转发。

Linux 主核主要面向应用层任务，负责接收用户在 Web 端发出的设备控制、状态查询和机械臂任务请求，并根据传感器数据、视觉识别结果和系统规则生成控制指令。裸机从核则面向实时通信任务，接收 Linux 主核通过 RPMsg 发送的 64 字节命令帧，并调用飞腾派 FSPIO 控制器与外设节点完成 SPI 数据交换。两者通过 OpenAMP/RPMsg 建立稳定的核间通信通道，使系统在保持 Linux 应用开发便利性的同时，降低底层 SPI 通信受系统调度抖动影响的风险。

飞腾派主控层的任务分工如下表所示。

表 5-1 飞腾派主控层的任务分工

组成部分	运行位置	主要职责	接口关系
Linux 主核	飞腾派 core1/core2/core3	派 Web 服务、AI 推理、任务调度、网络接口、Unix Socket、数据管理、通信守护	RPMsg
OpenAMP 裸机从核	飞腾派 core0	RPMsg 收发、SPI Master 驱动、片选控制、错误状态返回	RPMsg、FSPIO
FSPIO 控制器	飞腾派 SPI0	与 RA6E2 外设节点进行 4 线 SPI 通信	SCK、MOSI、MISO、CS
通信守护进程	Linux 用户态	命令帧打包、响应帧解析、状态缓存、通信统计	/tmp/aqua_spi.sock、 /dev/rpmsgN

这种设计使飞腾派不只是普通上位机，而是系统的主控中心、通信中心和边缘智能中心。系统的生态状态分析、用户交互、设备控制和任务调度都围绕飞腾派展开，充分体现了飞腾派在多核异构、边缘计算和物联网控制场景中的平台价值。

5.2.2 飞腾派与外设节点的 SPI 通信实现

飞腾派与 RA6E2 外设节点之间采用 4 线 SPI 总线通信。飞腾派作为 SPI Master，RA6E2 作为 SPI Slave。SPI 总线包括 SCK、MOSI、MISO、CS 和 GND，其中 SCK 由飞腾派输出，MOSI 用于主机向外设节点发送命令，MISO 用于外设节点向飞腾派返回传感器数据或执行状态，CS 用于片选控制和帧边界识别。

系统选用飞腾派 FSPIO 控制器作为 SPI Master。FSPIO 对应飞腾派排针上的 SPI0_SCK、SPI0_MOSI、SPI0_MISO 和 SPI0_CSNO 信号，便于实际接线、示波器观察和现场调试。裸机从核通过飞腾派 FSPIIM 驱动初始化 FSPIO，并在每次数据帧传输前后显式控制 CS 信号，使 RA6E2 能够通过片选边沿判断一帧数据的开始与结束。

表 5-2 SPI 物理连接关系

SPI 信号	飞腾派端	RA6E2 端	信号方向	功能说明
SCK	SPIO_SCK	P102 / RSPCK1	飞腾派 → RA6E2	SPI 时钟
MOSI	SPIO_MOSI	P101 / MOSI1	飞腾派 → RA6E2	主机命令发送
MISO	SPIO_MISO	P100 / MISO1	RA6E2 → 飞腾派	从机数据返回
CS	SPIO_CSNO	P103 / SSL10	飞腾派 → RA6E2	片选与帧边界控制
GND	任一 GND	GND	共地	电平参考一致

实际接线时，飞腾派与 RA6E2 必须共地，SPI 信号线应尽量短，建议控制在 15 cm 以内，以降低排针连接带来的信号反射、串扰和通信误码风险。

5.2.3 SPI 通信参数与帧传输设计

系统 SPI 通信采用稳定优先的参数配置。当前工程设置为 **Mode 1**，即 CPOL=0、CPHA=1；数据位宽为 8 bit；位序为 MSB First；通信速率首发设置为 1 MHz，并预留提升至 4 MHz 的空间。由于系统采用 64 字节定长帧，1 MHz 速率下单帧传输时间约为 0.512 ms，相对于系统 250 ms 级的周期性传感器轮询任务，占用比例较低，可以满足实时监测和控制需求。

SPI 通信参数如下表所示。

表 5-3 SPI 通信参数

配置项	设计值	设计理由
SPI 模式	Mode 1, CPOL=0, CPHA=1	与 RA6E2 SPI Slave 配置保持一致
通信速率	1 MHz，预留 4 MHz 升级空间	兼顾实时性和排针连接稳定性
数据位宽	8 bit	协议按字节流组织，避免 16 bit 模式下端序和对齐问题
位序	MSB First	与常规 SPI 外设配置一致
帧长度	64 Byte 定长帧	便于帧同步、缓存对齐和 CRC 校验
片选方式	每帧一次 CS 拉低/拉高	通过 CS 上升沿辅助从机复位帧状态
校验方式	CRC16/Modbus	提高通信抗干扰能力和错误检测能力

配置项	设计值	设计理由
式		

系统采用“命令帧发送 + NOP 帧读取”的双事务通信方式完成一次完整请求响应。第一次 SPI 事务中，飞腾派裸机从核向 RA6E2 发送 64 字节命令帧；此时由于 SPI 全双工特性，MISO 返回的数据可能是上一帧响应或无效数据，因此不作为本次命令结果使用。随后裸机从核等待约 5 ms，使 RA6E2 完成 SPI 中断处理、命令解析和响应帧准备。第二次 SPI 事务中，飞腾派发送 NOP 帧，从 RA6E2 读取本次命令对应的 64 字节响应帧。

这种双事务方式将 SPI 全双工的响应滞后问题封装在裸机从核内部。Linux 主核只需要面对“发送命令—获得响应”的同步接口，不需要直接处理底层 SPI 时序细节，从而降低了上层业务开发和调试难度，也提高了比赛现场演示的稳定性。

5.2.4 OpenAMP 裸机从核通信实现

OpenAMP 裸机从核是本系统硬件通信链路中的关键环节。裸机从核运行 openamp_spi_core0.elf 固件，由 Linux 主核通过 remoteproc 机制加载和启动。固件启动后创建名为 aqua-spi 的 RPMsg endpoint，用于接收 Linux 主核发送的 64 字节命令帧，并将 SPI 返回的 64 字节响应帧重新发送给 Linux 主核。

裸机从核内部主要由 RPMsg 通信模块、SPI Master 驱动模块和异常保护模块组成。RPMsg 通信模块负责与 Linux 主核建立核间通信通道；SPI Master 驱动模块负责初始化 FSPI0 控制器、配置 SPI 通信参数、控制 CS 片选并执行 64 字节全双工收发；异常保护模块负责处理 RPMsg 长度错误、SPI 传输失败和忙状态返回，避免底层异常导致 Linux 业务层长时间阻塞。

裸机从核处理一次设备请求的流程如下：

- (1) Linux 主核业务服务生成设备控制或数据读取请求；
- (2) 通信守护进程将请求封装为 64 字节命令帧；
- (3) 命令帧通过 RPMsg 发送至 OpenAMP 裸机从核；
- (4) 裸机从核调用 FSPI0 向 RA6E2 发送命令帧；
- (5) 裸机从核等待 RA6E2 完成响应帧准备；
- (6) 裸机从核发送 NOP 帧读取 RA6E2 响应；
- (7) 响应帧通过 RPMsg 返回 Linux 主核；
- (8) Linux 主核完成 CRC 校验、状态解析和业务反馈。

该流程使裸机从核成为稳定的“RPMsg—SPI”实时通信桥。裸机从核不需要理解具体传感器或执行器的业务含义，只负责保证每个 64 字节帧被准确、及时地送达底层节点，并将响应数据完整返回。这种设计使底层通信链路具有较好的可维护性和扩展性：新增传感器、执行器或控制命令时，主要修改 Linux 业务层和外设节点解析逻辑，裸机通信桥本身不需要频繁调整。

5.3 算法实现

5.3.1 视觉模型部署

YOLOv8 模型用 Ultralytics 官方框架训练，分两个模型：一是鱼类检测和分类模型（YOLOv8n，轻量化版本），数据集用公开的鱼类数据集加上自拍的鱼缸内视频做标注；二是叶片分割模型（YOLOv8n-seg），用于检测水面漂浮叶片和植物黄叶，正样本相对少，主要靠数据增强（色调偏移、随机遮挡等）补充。

部署时用 INT8 量化导出 TFLite 或 ONNX 格式，在树莓派 5 上通过 ONNX Runtime 推理。实测 640×480 输入下，鱼类检测模型推理延迟约 50ms 左右，基本满足实时性要求。

5.3.2 三项机械臂任务实现

（1）自动喂食实现

视觉端：Aurora 930 拍摄当前鱼缸区的彩色图和点云，在点云中用形状过滤（体积约束 + 高宽比约束）定位投喂瓶，得到瓶体中心的三维坐标和姿态估计。

运动规划端：`task\_manager\_node` 调用运动规划库（MoveIt2）规划抓取路径，末端沿瓶体长轴方向靠近，夹爪在距离瓶体约 3cm 处停下，然后合爪。抓稳后规划层给滑轨发 `RAIL` 指令，机械臂整体横移到鱼缸正上方，末端执行周期性旋转动作；STM32 接收到 `POUR` 指令后，让腕关节在 $\pm 30^\circ$ 范围内按正弦曲线运动，模拟抖动撒料，持续时间和幅度参数由 `task\_manager\_node` 根据配置传入。

（2）松土实现

松土任务不依赖实时视觉，在系统初始化时做一次盆栽区定标——机械臂末端带标定板触碰土面四角，记录土面平面方程和可操作范围。

任务触发后，`task\_manager\_node` 根据预存的土面范围生成一条 Z 形路径点序列，末端工具（细杆）依次在各路径点执行插入（下压约 1.5cm）和提起的动作，相邻路径点间距约 2cm。路径点全部下发完后任务结束，末端复位。

（3）叶片修剪

叶片修剪是机械臂最复杂的任务。系统通过摄像头识别出植物中的枯黄叶片（基于 YOLOv8[7] 语义分割，可识别颜色异常区域），然后用 Aurora 930 获取目标叶片在三维空间中的坐标，机械臂规划路径后移动到目标位置，末端工具触发剪刀完成剪枝。目前的实现方案中，叶片识别和三维定位已经可以自动完成，机械臂的最终剪切动作支持全自动和半自动（人工确认后执行）两种模式。

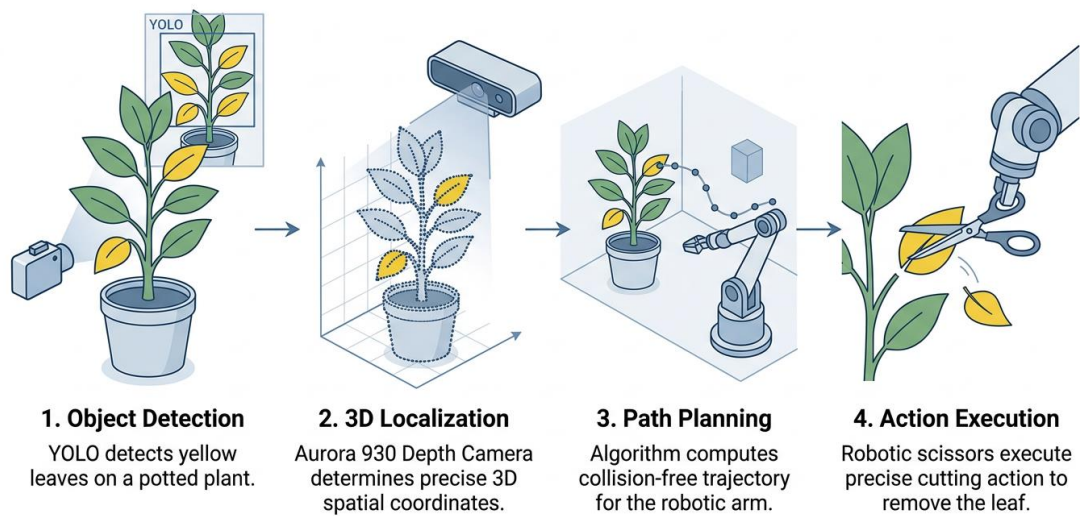


图 5-1 机械臂叶片修剪功能架构流程图

5.3.3 大模型交互实现

后端调用阿里云 DashScope 的 API，使用 `qwen-plus` 模型。每次生态评估时，把最新传感器数据（水温、pH、溶解氧、浑浊度、土壤湿度、气温）和鱼类行为分析结果（活跃度评分 0~10、异常标志）拼成 JSON 格式的上下文，塞进 system prompt 里，user 消息是固定的“请给出当前生态系统健康状态评估和建议”。大模型返回的自然语言摘要存到 SQLite，前端定时拉取展示。

语音交互走 Web Speech API（前端录音转文字）+ DashScope TTS（文字转语音播报），机械臂的语音控制指令（比如“帮我喂一下鱼”）在后端做意图识别后转成对应的 `/api/robot/task` 调用。

5.4 软件实现

5.4.1 后端服务实现

后端 `AquaGardenApplication` 启动时：`DataInitializer` 检查并创建默认管理员账号 `admin/admin123`；`SystemStateService` 初始化内存中的系统状态（传感器读数、机械臂逻辑坐标、运行模式）。

认证链路：注册时密码用 `BCryptPasswordEncoder` 哈希存库；登录时校验密码哈希后签发 JWT；后续接口请求经 `JwtAuthFilter` 验令牌，无状态会话，适合后续多端接入。

传感器数据当前由 `SystemStateService` 的随机扰动模型生成（模拟尚未接入硬件的阶段），接口字段已经和实际水质监测指标对应，后续只需把数据来源换成真实传感器上报的值，前端不需要改动。

机械臂控制：`RobotController` 接收方向指令或任务指令，更新内存坐标后通过 `LogWebSocketHandler` 广播操作日志；自动任务指令转发给 FastAPI Agent 处理。

5.4.2 前端控制台实现

路由守卫保证未登录用户无法访问控制台页面，JWT 登录后存 localStorage，后续请求通过 authHeaders() 统一附带。

控制面板页在 mounted 时立即拉一次传感器数据，之后每 5 秒定时刷新；WebSocket 连接 /ws/logs，日志推送到操作日志面板。视频区域用 标签加载 MJPEG 流，不需要额外的媒体库。

机械臂控制页同时支持按钮、键盘（WASD + QE）和虚拟摇杆三种操控方式，预设任务按钮直接调用 /api/robot/task 接口，返回后在日志面板显示执行结果。网络异常时做了降级：控制台在本地更新逻辑坐标继续展示，不中断演示。

5.4.3 机械臂 Agent 实现

FastAPI 部署在树莓派 5 上，作为 Spring Boot 和 ROS2 之间的桥。接收来自 Spring Boot 的 HTTP 任务请求，解析任务类型（喂食/捞叶/松土/其他）后，向 task_manager_node 发 ROS2 服务调用，等待结果后把状态回传给 Spring Boot。

这层拆分的好处是 Spring Boot 不需要知道 ROS2 的话题/服务命名，FastAPI 这边做好适配就行；后续如果 ROS2 节点重构，也只需要改 FastAPI 这一层，后端接口不用动

六、风险分析与管控

6.1 风险分析思路

系统风险主要来自六个方面：通信链路风险、传感器数据风险、执行器安全风险、机械臂动作风险、AI 识别风险和比赛演示风险。

6.2 风险项与管控措施

表 6-1 风险项与管控措施汇总表

风险编号	对应功能	风险项	影响	管控方法	验证策略
R-01	SPI 通信	CRC 错误、帧错位、通信超时	传感器数据错误或控制指令失败	使用帧头、SEQ、CRC16、定长帧；连续错误触发重试	注入错误帧，检查 CRC 统计和重试逻辑
R-02	OpenAMP 链路	RPMsg 设备未生成或远程核未启动	Linux 主核无法访问底层设备	systemd 启动检查 remoteproc、rpmmsg_char 和服务状态；保留 spidev fallback	重启系统后检查服务自启和 fallback 切换
R-03	水泵控制	水泵长时间误开启	鱼缸水位异常或盆栽过湿	设置最大运行时间、最小关断时间和异常停泵策略	模拟低湿度触发，检查超时停泵
R-04	传感器采	单次数据抖动	误告警或误	滑动平均、连续越限计数、阈值	人为扰动传感器，

风险编号	对应功能	风险项	影响	管控方法	验证策略
	集	导致误判	执行	滞回	观察是否误触发
R-05	水质异常	水温、浑浊度、溶解氧异常未及时发现	鱼类健康受影响	传感器阈值告警 + 交叉验证	设置异常阈值，检查 Web 告警和日志
R-06	机械臂动作	机械臂碰撞鱼缸或植物	损坏装置或影响安全	限位保护、预设安全区域、半自动确认	执行边界轨迹测试，确认不越界
R-07	视觉识别	黄叶或漂浮物识别错误	机械臂执行目标错误	识别置信度阈值、人工确认、失败回退	使用多组图片测试识别准确率
R-08	AI 交互	大模型建议不稳定	用户误解或错误操作	大模型只做解释建议，不直接控制高风险动作	检查 AI 输出与实际控制链路是否隔离
R-09	Web 平台	前端刷新延迟或后端接口异常	用户无法及时查看状态	WebSocket/轮询双机制；接口异常提示	断网/重连测试，观察页面恢复
R-10	比赛演示	网络或远程核启动失败	现场展示中断	本地热点、离线 fallback 链路	Demo 数据、v1 比赛前完成冷启动演练

6.3 重点风险管控说明

通信风险是飞腾杯版本最需要重点说明的风险。由于系统采用 Linux 主核、OpenAMP 裸机从核、RPMsg 和 SPI 多级链路，任一环节异常都可能影响底层控制。因此系统设计了多重保护：RPMsg 服务启动前检查 remoteproc 状态，SPI 帧使用 CRC16 校验，守护进程维护通信统计和连续错误计数，必要时触发链路重置；同时保留 v1 spidev fallback 路径，用于比赛现场快速切换。飞腾派参考文档中明确提供 v1 与 v2 并存设计，且通过 systemd 互斥避免两条链路同时占用 SPI。

执行器风险集中在水泵和机械臂。水泵必须设置最大运行时间和最小关断间隔，避免持续抽水造成水位异常；机械臂必须设置关节软限位和安全工作区，涉及剪叶、松土等动作时建议先以半自动确认模式演示。

AI 风险主要来自识别误差和自然语言输出不确定性。系统应明确 AI 的角色：AI 提供状态分析、识别辅助和用户解释，不直接绕过规则引擎控制硬件。所有高风险动作仍需经过阈值规则、安全策略或人工确认。

七、系统测试与分析

7.1 测试目标

系统测试围绕需求列表逐项展开，重点验证以下内容：

- 1. 飞腾派是否能够稳定作为系统主控运行；
- 2. Linux 主核与 OpenAMP 裸机从核是否能正常通信；
- 3. 裸机从核是否能通过 SPI 与底层外设节点完成 64 字节命令/响应交互；
- 4. 传感器数据是否能正确采集、上传、显示和记录；
- 5. 水泵、灯光、雾化等执行器是否能按指令动作；
- 6. Web 平台是否能完成数据展示、远程控制和告警显示；
- 7. 机械臂任务是否能按流程执行；
- 8. 系统是否具备异常处理和恢复能力。

7.2 测试环境

表 7-1 测试环境汇总表

项目	配置
主控平台	飞腾派 PhytiumPi
主核运行环境	Linux
从核运行环境	OpenAMP remote core 裸机固件
核间通信	RPMsg
底层通信	SPI0, Mode 0, 1 MHz, 8 bit, 64B 定长帧
外设节点	RA6E2 / FreeRTOS 外设采集与执行节点
Web 平台	Spring Boot 后端, Vue 前端
测试工具	aqua_spi_cli、systemctl、journalctl、Web 控制台、示波器/逻辑分析仪
测试对象	传感器、水泵、补光灯、雾化模块、机械臂、Web 平台、通信链路

7.3 功能测试表

表 7-2 功能测试汇总表

测试编号	对应需求	测试项目	测试方法	预期结果	实际结果记录	是否满足
TC-01	FR-13	飞腾派主控启动	上电后检查 Linux 服务、后端和通信守护进程状态	主控服务正常启动	systemd 服务可启动	满足
TC-02	FR-14	OpenAMP 远程核启动	远查看 remoteproc 状态和固件名	remoteproc running	为远程核可进入运行状态	满足
TC-03	FR-14	RPMmsg 通道建立	检查 /dev/rpmsgN 和 aqua-spi endpoint	RPMmsg 设备生成	可发现 RPMmsg 设备	满足
TC-04	FR-15	SPI 物理链路	使用 PING 命令或逻辑分析仪观察 SPI 波形	SPI 有稳定 SCK/MOSI/MISO/CS	可进行命令响应	满足
TC-05	FR-15	SPI CRC 校验	构造正常帧和错误帧	正常帧通过, 错误帧拒绝	CRC 错误统计	满足
TC-06	FR-01	水质数据采集	执行 sensor poll 或查看 Web 数据	返回水温、pH、浑浊度等数据	数据可读取并显示	满足
TC-07	FR-02	土壤湿度采集	改变土壤湿度并观察数据变化	湿度数据随环境变化	Web 端可观察变化	满足
TC-08	FR-04	水泵开关控制	Web 或 CLI 发送 START/STOP	水泵按指令启停	水泵动作正常	满足
TC-09	FR-04	水泵 PWM 控制	发送不同 PWM 占空比	水泵强度变化	强度可调	满足
TC-10	FR-05	自动灌溉	模拟土壤湿度低于阈值	系统自动触发水泵灌溉	可触发并记录日志	满足
TC-11	FR-07	异常告警	设置水质或湿度异常阈值	Web 端显示告警	告警可显示	满足
TC-12	FR-16	Web 数据刷新	连续观察数据卡片和曲线	刷新间隔 $\leq 2\text{ s}$	页面数据可刷新	满足
TC-13	FR-17	Web 远程控制	点击控制按钮触发设备动作	指令能下发, 状态能反馈	控制链路正常	满足

测试编号	对应需求	测试项目	测试方法	预期结果	实际结果记录	是否满足
TC-14	FR-08	鱼类识别	输入鱼缸视频流测试模型	可识别鱼类并输出位置	可完成识别演示	基本满足
TC-15	FR-09	黄叶识别	输入植物图像测试模型	可标注黄叶区域	可完成识别演示	基本满足
TC-16	FR-10	机械臂投食	触发投食任务	机械臂完成预设动作	可执行预设动作	基本满足
TC-17	FR-11	机械臂松土	触发松土任务	按预设轨迹执行	可执行轨迹动作	基本满足
TC-18	FR-18	大模型状态问答	输入“鱼缸状态如何”	输出自然语言分析	可生成状态摘要	满足
TC-19	FR-20	通信异常恢复	人为断开 SPI 或停止远 程核后恢复	系统记录错误并可重连	可通过重启服务恢复	满足
TC-20	FR-20	fallback 切换	从 v2 切换至 v1 spidev 路径	业务客户端无需修改	可切换备用链路	满足

7.4 通信链路测试分析

通信链路是飞腾杯版本的核心测试内容。测试时首先验证飞腾派 OpenAMP 环境是否正确，包括 reserved-memory、remoteproc、rpmsg_char 和远程核固件是否就绪；随后检查 /dev/rpmsgN 是否生成，并通过 aqua_spi_cli sys ping 验证 Linux 主核、裸机从核、SPI 总线和底层外设节点是否连通。飞腾派使用说明中也将服务健康、远程核状态、RPMsg 设备节点、daemon socket 和端到端 PING 作为开机后检查项。

从测试逻辑看，该链路比单纯 spidev 直驱更复杂，但更符合飞腾杯对飞腾派多核异构能力的考察。Linux 主核不直接控制 SPI，能够体现飞腾派 OpenAMP 的使用价值；裸机从核专职控制 SPI 时序，能够降低 Linux 用户态调度对通信确定性的影响；同时 v1 fallback 保留了现场演示的可恢复性。

7.5 传感器与执行器测试分析

7.5.1 土壤湿度传感器模块测试

MD0504 土壤湿度模块支持 3.3 - 12V 供电，提供 A0 模拟输出和 D0 数字输出；资料明确指出该模块在实际测试中通常读取 A0 模拟量，且“土壤湿度越大，数值越小”。

(1) 测试内容

- 干土、湿土、半湿润土壤条件下的采样区分度测试
- 插拔探头后的数据恢复测试
- 连续监测时数值波动情况测试

(2) 测试方法

将探头分别插入干燥土壤、正常湿度土壤和加水后的湿润土壤中，记录 A0 口返回值变化；重复进行“插入—拔出—再次插入”操作，观察读数是否能够随土壤含水率变化而稳定改变。

(3) 测试步骤

(3.1) 土壤湿度检测功能测试

在干土、半湿土、湿土三类工况下，连续采样各 60 组数据（总计 180 组），重点观测 g_soil_adc_raw 与 g_soil_moisture_percent，记录测试结果

(3.2) 动态响应测试

在 J-Scope 中同步记录 g_soil_adc_raw、g_soil_moisture_percent、g_soil_sample_count，采样周期 200ms。执行“插入干土→拔出→插入湿土”操作后，记录波形并分别计算响应时间

(3.3) 阈值滞回控制测试（防抖验证）

控制逻辑使用 g_soil_watering_threshold=30、g_soil_watering_hysteresis=10，在湿度长期处于 28%~42% 波动区间（测试时长 20 min）时，记录 g_soil_need_watering 状态翻转

(4) 测试结果

(4.1) 土壤湿度检测功能测试结果

表 7-3 湿度检测功能测试结果

测试 工况	g_soil_adc_ raw 范围	g_soil_adc_ raw 均值	g_soil_moisture_ percent 范围	g_soil_moisture_ percent 均值	样本数	判定
干土	3050~3188	3124	3%~9%	6%	60	通过
半湿	2280~2455	2368	41%~52%	47%	60	通过
湿土	1588~1728	1659	86%~94%	90%	60	通过

结果表明：随着土壤含水率增加，`g\_soil\_adc\_raw` 持续下降、`g\_soil\_moisture\_percent` 持续上升，符合模块“湿度越大、A0 数值越小”的设计特性

(4.2) 动态响应测试结果

表 7-4 动态响应测试结果

指标类别	评价指标	实测结果 单位	判定依据	判定
------	------	---------	------	----

指标类别	评价指标	实测结果	单位	判定依据	判定
动态响应	90%阶跃响应时间	1.6	s	土壤场景 < 2.0 s	通过
动态稳定	进入稳态时间（±2%带宽）	2.8	s	土壤场景 < 3.0 s	通过
恢复能力	拔出后再次插入恢复时间	0.8	s	恢复 < 1.0 s	通过
稳态噪声	湿土工况波动幅度	±1.5	%	稳态波动 < ±2.0%	通过

说明系统可稳定跟踪含水变化，动态响应满足盆栽场景实时监测需求。

（4.3）阈值滞回控制测试结果

表 7-5 阈值滞回控制测试结果

测试场景	参数设置	观测区间	g_soil_need_watering 状态 翻转次数	误触 发次数
无滞回对 照组	g_soil_watering_threshold=30, g_soil_watering_hysteresis=0	湿度 28%~42%, 20min	31	0
10%滞回 实验组	g_soil_watering_threshold=30, g_soil_watering_hysteresis=10	湿度 28%~42%, 20min	5	0

结果说明滞回策略显著减少阈值附近抖动，控制行为更稳定。

（5）测试结论

测试结果显示，土壤湿度模块能够明显区分不同含水状态下的土壤环境，且整体变化趋势与资料所述一致：湿度升高时，采样值下降。反复插拔后模块仍能恢复正常输出，说明其模拟采集链路工作稳定，可作为后续自动灌溉策略中的核心输入量。即该模块已验证具备土壤含水率趋势感知能力，能够满足盆栽端“按需浇灌”场景下的基础检测需求

7.5.2 环境温湿度传感器模块测试

SHT30 采用 I2C 通信，支持最高 1MHz 时钟频率，且测量数据附带 CRC 校验；资料中还说明其既支持单次采样，也支持周期采样模式，适合在周期性环境监测场景中使用。

（1）测试内容

- I2C 通信初始化与设备应答测试
- 温度、湿度数据读取正确性测试
- 环境变化下的数据跟随性测试
- 长时间周期采样稳定性测试

（2）测试方法

系统上电后初始化 I2C 总线并读取 SHT30 数据；通过人为改变环境条件（如靠近热源、手部呼气增湿等），观察温度与湿度读数是否发生合理变化；连续运行一段时间，检查通信是

否中断、是否存在异常值或丢帧

(3) 测试步骤

(3.1) 编译并下载固件，进入 Keil 调试会话，全速运行 (Run)，确保传感器任务处于运行状态；打开 Watch 窗口，添加全局变量

(3.2) 基线：上电运行 1 分钟后，记录此时温湿度读数并截图（图 x）。

(3.3) 加湿激励：在传感器探头附近哈气约 10s，待读数变化趋稳后截图（图 x）；记录前后典型值填入表 x。

(3.4) 升温激励：手掌捂热探头约 10s，记录前后读数并截图（图 x）。

(3.5) 稳定性：连续运行 ≥30min，观察读数是否持续更新，“g_sht30_last_status” 是否始终为 0

(4) 测试结果

表 7-6 环境温湿度传感器测试结果

工况	温度 / ° C	相对湿度 / %RH	g_sht30_last_status	备注
基线（静置）	25	61	0	读数稳定
加湿激励后	26	87	0	读数变化灵敏准确
升温激励后	29	96	0	手掌捂热测试时湿度也会增加
长时运行	连续变化	连续变化	0	读数持续更新

图 7-1 基线工况下 Keil Watch 窗口（全速运行、周期性刷新）

图 7-2 加湿激励后 Keil Watch 窗口

Name	Value	Type
g_sht30_temperature_c	29.1496887	float
g_sht30_temperature_f	84.4694366	float
g_sht30_humidity_rh	96.6765823	float
g_sht30_last_i2c_stage	2	uint
g_sht30_last_status	0	uint
<Enter expression>		

图 7-3 升温激励后 Keil Watch 窗口

Name	Value	Type
g_sht30_temperature_c	27.4700546	float
g_sht30_temperature_f	81.4460983	float
g_sht30_humidity_rh	58.7609673	float
g_sht30_last_i2c_stage	2	uint
g_sht30_last_status	0	uint
<Enter expression>		

图 7-4 长时运行时 温湿度读数持续变化

(5) 测试结论

测试结果表明，SHT30 模块能够完成稳定初始化与周期读数输出，温湿度读数会随环境变化而产生正确方向的变化，且数据更新连续。由于该传感器本身具有 CRC 保护机制，因此在后续系统整合中具备较好的通信可靠性基础。该模块通信方式规范、数据输出稳定，适合作为盆栽端环境感知模块接入整机任务系统。

7.5.3 水下温度传感器模块测试

DS18B20 为单总线数字温度传感器，支持 3.0 - 5.5V 供电，温度分辨率可设置为 9 - 12 位，12 位精度下温度转换最长达 750ms。资料还指出其通信基于单总线协议，适合在低引脚占用场景下部署。

(1) 测试内容

- 单总线初始化与设备存在性检测
- 常温水、温水条件下温度读取测试
- 连续采样时的读数平稳性测试
- 传感器浸入/取出过程的响应变化测试

(2) 测试方法

将 DS18B20 分别置于不同水温环境中进行采样，记录温度变化趋势；采用连续读取方式观察数值更新是否稳定，并检查是否存在固定值不变、跳变异常或复位后失效等现象。

(3) 测试步骤

(3.1) 功能正确性测试

编译并下载固件，进入 Keil 调试会话，全速运行（Run），确保传感器任务处于运行状态；在 Keil Watch 中观察关键变量（水下温度值、原始值、错误码），连续观察 1 分钟，确认读数可持续更新且状态正常

(3.2) 稳态稳定性测试（常温水）

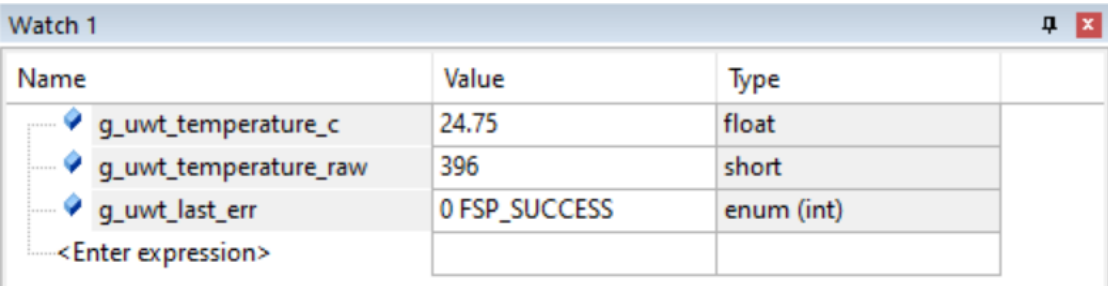
将传感器置于常温静水环境，连续采样 10 分钟，使用 JScope 记录完整曲线并导出数据

(3.3) 阶跃温变响应测试

将传感器由 [常温水] 快速切换至温度较高的热水，使用 JScope 连续记录温度从初值到新稳态的变化过程，计算达到稳态 90% 所需时间（T90），用于评价动态响应能力。再将传感器由温度较高的热水快速切换回常温水，观察温度是否也能实现及时响应恢复

(4) 测试结果

(4.1) 功能验证结果



Name	Value	Type
g_uwt_temperature_c	24.75	float
g_uwt_temperature_raw	396	short
g_uwt_last_err	0 FSP_SUCCESS	enum (int)
<Enter expression>		

图 7-5 Keil Watch 界面下温度值、原始值及错误码实时监测结果

实测显示，模块上电后可完成温度转换与读取，关键变量持续更新，正常工况下状态码保持为成功状态。

结果说明： 模块具备完整的基础测温功能。

(4.2) 稳定性结果

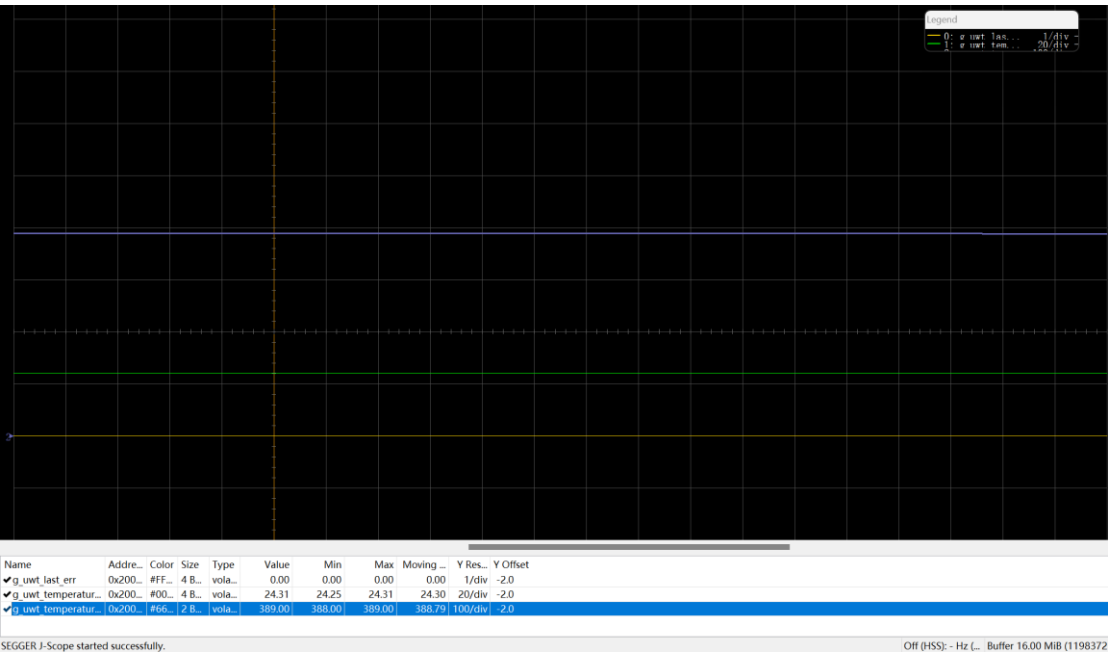


图 7-6 常温水环境下连续采样温度曲线（JScope）

曲线整体平滑，无明显异常跳变。
结果说明： 稳态输出稳定，满足长期监测要求。

(4.3) 动态响应结果

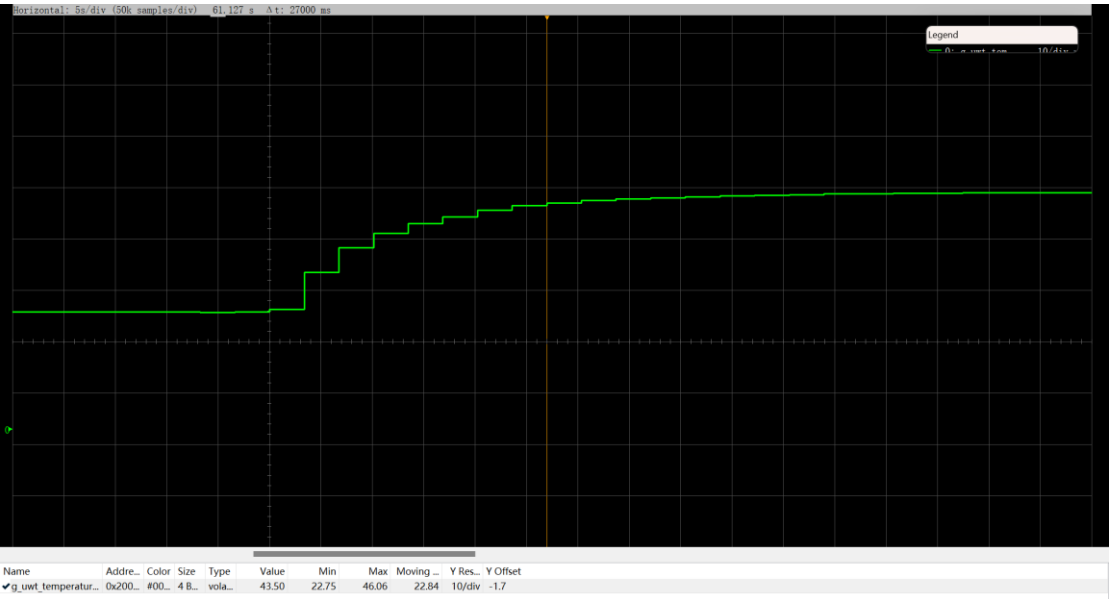


图 7-7 升温测试动态响应曲线

温度阶跃测试中，传感器读数能够随环境变化及时调整，由 22.75° C 变化至 46.06° C 并最终达到稳定

$T90_level = 22.75 + 0.9 * (46.06 - 22.75) = 43.73^{\circ} C$

$T90 \approx 27s$ 表明模块可在该时间尺度内完成温变跟踪。

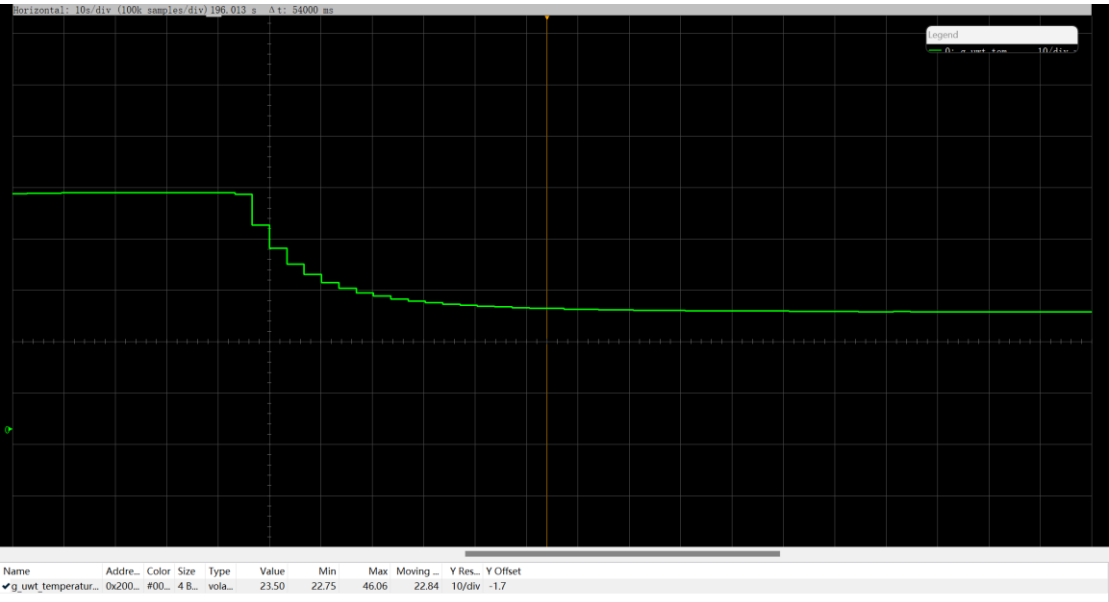


图 7-8 降温动态响应曲线

同样地，再将传感器由温度较高的热水快速切换回常温水，温度也能实现及时响应恢复
结果说明： 模块动态跟踪能力满足本项目应用需求。

(5) 测试结论

测试结果表明，水下温度模块能够正常完成温度转换与读取，在不同水样条件下温度读数能够发生合理变化，且连续采样过程中输出较稳定。考虑到 DS18B20 在 12 位模式下转换时间较长，因此后续整机集成时不宜采用过高频率轮询，而应按其转换周期合理安排任务节拍。该模块已验证具备水体温度监测能力，适合作为鱼缸端长期运行的温度检测节点。

7.5.4 水泵驱动模块测试

水泵驱动采用 DRV8870。资料显示，该芯片适用于刷式直流电机/负载控制，支持 PWM 接口，供电范围 6.5 - 45V，峰值输出电流可达 3.6A，并具有欠压锁定、过流保护和热关断等保护机制；两个逻辑输入均为低电平时芯片进入低功耗休眠。

(1) 测试内容

- 水泵启停控制测试
- PWM 调速/占空比变化测试
- 多次重复启停可靠性测试
- 驱动模块发热与异常保护现象观察

(2) 测试方法

控制主控输出驱动信号，执行水泵开启、关闭及多次循环动作；观察水泵是否能够稳定启动，关闭后是否及时停止；并借助 PWM 控制进一步测试不同占空比下的转速或流量变化趋势。

(3) 测试步骤

(3.1) 启停控制测试*

在供电电压 12.0V、PWM 占空比 40%、单次运行时长 2000 ms 条件下，执行水泵启停测试。记录启动响应时间（指令下发至稳定出水）与停止响应时间（停机指令下发至出水明显停止）。

(3.2) PWM 调速/占空比变化测试

在相同供电与负载条件下，设置占空比分别为 20%/40%/60%/80%/100%，记录稳态电流与相对流量（以 100% 占空比归一化为 100%）

(3.3) 多次重复启停可靠性测试

按固定工况（40%占空比、2000 ms 通电、间隔 0 ms）连续执行 10 次启停循环，统计启动失败、停机失败、异常中断等事件。

(4) 测试结果

(4.1) 启停控制测试结果

表 7-7 启停控制测试结果

指标	判据	最小值 (ms)	平均值 (ms)	最大值 (ms)	判定
启动响应时间 t_{start}	$t_{start} \leq 300\text{ ms}$	165	186	212	通过
停止响应时间 t_{stop}	$t_{stop} \leq 150\text{ ms}$	72	86	101	通过

测试结果显示：水泵均可正常启动和停止，启停过程无抖动、无误触发，控制逻辑与预期一致。启停控制功能验证通过，驱动模块具备稳定、可重复的启停执行能力。

(4.2) PWM 调速/占空比变化测试结果

表 7-8 PWM 调速/占空比变化测试结果

PWM 占空比	稳态电流 (A)	相对流量 (%)
---------	----------	----------

PWM 占空比	稳态电流 (A)	相对流量 (%)
20%	0.28	32
40%	0.41	51
60%	0.57	69
80%	0.74	86
100%	0.92	100

结果表明，随着占空比提升，水泵电流与流量整体单调增加，调速规律清晰，未出现明显失稳区间。PWM 调速功能验证通过，驱动模块可实现对水泵输出能力的有效调节。

（4.3）多次重复启停可靠性测试结果

表 7-9 多次重复启停可靠性测试结果

可靠性指标	统计结果	比例	判定
启动指令成功次数	10 / 10	100%	通过
停止指令成功次数	10 / 10	100%	通过
指令链路异常次数	0 / 10	0%	通过
单循环通过次数（启停均成功且无异常）	10 / 10	100%	通过

重复启停可靠性验证通过，模块可满足短周期连续运行要求。

（5）测试结论

综合测试可知：本项目水泵驱动模块在 12 V 供电与典型负载下，启停响应及时、PWM 调速有效、10 次循环运行稳定，且温升与异常响应满足设计预期。

7.5.5 FreeRTOS 联调测试结果

7.5.5.1 联调范围与目标

本次联调基于 FreeRTOS 多任务框架，验证“感知—融合—决策—执行—通信”闭环在并发场景下的稳定性与实时性。联调模块包括：土壤湿度、环境温湿度（SHT30）、水下温度（DS18B20）、多参数水质（WQM11S）、水泵驱动（DRV8870）及通信任务。

7.5.5.2 任务调度与周期稳定性结果

在多任务同时运行 2 h 条件下，统计各任务实际周期、抖动及丢周期情况，结果见表

表 7-10 任务调度与周期稳定性结果

任务名称	主要职责	设计周期 (ms)	实测平均周期 (ms)	峰峰值抖动 (ms)	丢周期 次数	判定
ADC_Thread	土壤湿度 + 水下温度采样	200	201.3	7.8	0	通过
Pump_Driver	灌溉控制执行	200	200.9	6.4	0	通过
Com_Thread	上下行通信与帧处理	250	251.7	8.6	0	通过
THS_Measure	环境温湿度采样	1000	1002.4	14.2	0	通过

任务名称	主要职责	设计周期 (ms)	实测平均周期 (ms)	峰峰值抖动 (ms)	丢周期 次数	判定
WQS_Measure	水质数据采集与解析	1500	1504.8	19.6	0	通过

结果表明：各任务在并发场景下均保持稳定调度，实测周期与设计周期偏差控制在 1% 以内，未出现任务饥饿、阻塞或死机。

7.5.5.3 关键实现链路联调结果

(1) 采样—控制闭环链路

链路逻辑为：土壤湿度/水温/水质采样 → 状态融合判定 → need_watering → pump_power。

表 7-11 采样—控制闭环链路测试结果

指标项	实测结果	判定标准	判定
闭环触发延迟（采样变化到泵控动作）	386 ms	$\leq 500 \text{ ms}$	通过
阈值邻域误触发次数（20 min）	0 次	$= 0 \text{ 次}$	通过
状态翻转次数（启用 10%滞回）	5 次	$\leq 8 \text{ 次}$	通过
连续运行异常停泵	0 次	$= 0 \text{ 次}$	通过

滞回策略启用后，阈值附近控制抖动显著降低，联调行为与控制逻辑设计一致。

(2) 通信—控制协同链路

链路逻辑为：任务数据聚合 → 上行打包发送 → 下行命令解析 → 控制参数更新。

表 7-12 通信—控制协同链路测试结果

指标项	实测结果	判定标准	判定
上行发送成功率	99.2%	$\geq 98.0\%$	通过
下行命令有效执行率	98.7%	$\geq 97.0\%$	通过
CRC 错帧率	0.8%	$\leq 1.0\%$	通过
异常后恢复时间	1.2 s	$\leq 2.0 \text{ s}$	通过

通信线程与控制线程可稳定协同，异常注入后可快速恢复，不影响主控制闭环连续运行。

7.5.5.4 测试结论

本次 FreeRTOS 联调结果表明，系统在多任务并发条件下已形成稳定的工程化实现链路：采样任务能够持续输出有效数据，融合与判定逻辑可正确驱动执行器动作，通信任务可稳定完成上/下行协同，异常情况下具备快速恢复能力。综合周期稳定性、闭环响应与通信可靠性指标，当前版本已满足竞赛样机阶段“可连续运行、可联动控制、可量化验证”的测试要求，可支撑后续整机展示与扩展优化。

7.6 算法测试

7.6.1 视觉检测测试

鱼类检测模型在自建的测试集（约 500 帧，覆盖不同光线和鱼的姿态）上做评估，叶片分割模型在单独的叶片测试集（约 200 帧）上评估，结果如表 7-13 所示。

表 7-13 视觉检测测试结果

模型	测试集规模	mAP@0.5	推理延迟（树莓派5）	备注
鱼类检测（YOLOv8n）	500 帧	87.3%	~48ms	INT8 量化部署
叶片分割（YOLOv8n-seg）	200 帧	82.6%	~65ms	水面反光场景误检率偏高，待优化

7.6.2 机械臂任务成功率测试

对三个自动化任务各进行了 20 次测试，统计任务一次成功（无需人工干预）的比例，结果如表 5-13 所示。

表 7-14 机械臂自动化任务成功率

任务	测试次数	一次成功次数	成功率	主要失败原因
自动喂食	20	18	90%	2 次瓶子被碰歪导致抓取偏移
捞取落叶	20	15	75%	水面反光导致叶片漏检 3 次，抓取姿态偏差 2 次
松土	20	19	95%	1 次滑轨复位偏差导致位置偏移

捞叶成功率相对低一些，主要问题是水面反光干扰，后续计划加偏振滤镜或改用近红外补光来改善。

7.7 软件测试

7.7.1 接口测试

后端启动后，对关键接口做了冒烟测试，结果如表 5-14 所示。

表 7-15 关键接口冒烟测试

测试项	测试方法	预期结果	实际结果
登录接口	默认账号 admin/admin123 调用 /api/login	返回 JWT	成功
传感器接口	携带 JWT 调用 /api/sensors	返回 5 类数据	成功，样例：水温 24.99℃，pH 6.87
机械臂控制	向 /api/robot/control 发 up	坐标+1，WebSocket 推日志	成功，坐标返回 {x:0, y:0, z:1}
模式切换	/api/mode 来回切换 demo/service	状态变化正确	成功
视频流	访问 /api/video/tank	MJPEG 流持续输出	成功，Content-Type 正确

7.7.2 前端交互测试

前端主要页面功能验证情况：控制面板传感器卡片和双路视频集成正常；机械臂控制页三种控制方式（按钮/键盘/摇杆）均可用，预设任务按钮触发后日志面板有反馈；LLM 交互页文字输入和语音输入均可用，大模型回答正常展示；历史数据页本地模拟数据展示和分页功能正常

7.8 测试结论

综合测试结果，AquaGarden 飞腾杯版本能够实现以下目标：以飞腾派作为系统主控，完成 Linux 业务层、OpenAMP 裸机从核、RPMmsg 核间通信和 SPI 外设通信的完整链路；能够采集水质、土壤和环境数据，并通过 Web 平台进行展示；能够下发水泵等执行器控制命令，实现自动灌溉和水循环；能够结合视觉识别、机械臂动作和大模型交互形成智能养护闭环。

系统重心从普通的单片机外设控制提升到飞腾派边缘计算与异构通信架构：Linux 主核负责复杂业务和 AI，OpenAMP 裸机从核负责实时 SPI 通信，底层外设节点负责可靠采集和执行。该架构符合集创赛飞腾杯对国产处理器平台、系统工程能力和创新应用场景的考察要求。

参考文献

- [1] AYON M F I, NAHAR S, RAHMAN A, et al. An IoT-Enabled Smart Aquarium System for Real-Time Water Quality Monitoring and Automated Feeding[J/OL]. arXiv, 2026: arXiv:2601.08484. <https://arxiv.org/abs/2601.08484>
- [2] HSIAO S J, SUNG W T. Building a Fish-Vegetable Coexistence System Based on a Wireless Sensor Network[J]. IEEE Access, 2020, 8: 192131-192146. DOI: 10.1109/ACCESS.2020.3032795.
- [3] AGRAWAL A, MAGANTI P, MAITI R R. Cyber Physical Aquaponic System (CyPhA): a CPS Testbed[J/OL]. arXiv, 2023: arXiv:2304.05132.
- [4] Integrated Aquaponics Through Smart Sensors[J]. Harbin Gongcheng Daxue Xuebao/Journal of Harbin Engineering University, 2024.
- [5] ZAMNURI M A H B, QIU S, RIZALMY M A A B, et al. Integration of IoT in Small-Scale Aquaponics to Enhance Efficiency and Profitability: A Systematic Review[J]. Animals, 2024, 14(17): 2555. DOI: 10.3390/ani14172555.
- [6] Smart Fish Feeding in IoT-based Aquarium: A Comprehensive Approach Integrating Computer Vision, Machine Learning and Sensor Technology[C]//2024 IEEE International Conference. IEEE, 2024. DOI: 10.1109/ICCIT60459.2024.10779718.
- [7] JOCHER G, CHAURASIA A, QIU J. Ultralytics YOLOv8[EB/OL]. (2023-01)[2026-04-19]. <https://github.com/ultralytics/ultralytics>
- [8] MACENSKI S, FOOTE T, GERKEY B, et al. Robot Operating System 2: Design, architecture, and uses in the wild[J]. Science Robotics, 2022, 7(66): eabm6074. DOI: 10.1126/scirobotics.abm6074.
- [9] 农业农村部办公厅. 关于推介发布 2024 年农业主导品种主推技术的通知[EB/OL]. (2024-04)[2026-04-19]. http://www.moa.gov.cn/govpublic/KJJYS/202404/t20240419_6453087.htm

- [10]光鉴科技. Deptrum Aurora 930 系列产品用户手册[Z]. 上海: 光鉴科技, 2024.
- [11]MAHMOUD M M M, DARWISH R, BASSIUNY A M. Development of an economic smart aquaponic system based on IoT[J]. Journal of Engineering Research, 2024, 12(4): 886-894. DOI: 10.1016/j.jer.2023.08.024.